

UNIVERSITI TEKNOLOGI MARA

**DEEPPFAKE DETECTION IN AUDIO
USING MACHINE LEARNING**

HARIZ IMZAN BIN HUZEY IMZAN

**Bachelor of Information Systems (Hons.) Intelligent
Systems Engineering.**

JUNE 2024

UNIVERSITI TEKNOLOGI MARA

**DEEPPFAKE DETECTION IN AUDIO
USING MACHINE LEARNING**

HARIZ IMZAN BIN HUZEY IMZAN

**Final Year Project submitted in fulfilment of the
requirements for Bachelor of Information Systems
(Hons.) Intelligent Systems Engineering**

JUNE 2024

SUPERVISOR APPROVAL

DEEPPFAKE DETECTION IN AUDIO USING MACHINE LEARNING

By

**HARIZ IMZAN BIN HUZEY IMZAN
2023165037**

This thesis was prepared under the supervision of the project supervisor,
SHAMIMI A. HALIM (DR.). It was submitted to the Faculty of Computer and
Mathematical Sciences and was accepted in partial fulfilment of the requirements for the
degree of Bachelor of Information Systems (Hons.) Intelligent Systems Engineering.

Approved by

.....

SHAMIMI A. HALIM (DR.)

Project Supervisor

June 19, 2024

STUDENT DECLARATION

I certify that this thesis and the project to which it refers is the product of my own work and that any idea or quotation from the work of other people, published or otherwise are fully acknowledged in accordance with the standard referring practices of the discipline



.....

HARIZ IMZAN BIN HUZEY IMZAN

2023165037

June 19, 2024

ACKNOWLEDGEMENT

Alhamdulillah, praises and thanks to Allah because of His Almighty and His utmost blessings, I was able to finish this research within the time duration given. Firstly, my special thanks goes to my supervisor, SHAMIMI A. HALIM (DR.) Special appreciation also goes to my beloved parents. Last but not least, I would like to give my gratitude to my dearest friends

TABLE OF CONTENTS

CONTENT	Page
ACKNOWLEDGEMENT	III
TABLE OF CONTENTS	IV
LIST OF TABLES	VII
LIST OF FIGURES	VIII
LIST OF ABBREVIATIONS	IX
CHAPTER ONE: INTRODUCTION	1
1.1 Research Background	1
1.2 Problem Statement	3
1.3 Research Questions	4
1.4 Research Objectives	5
1.5 Research Scope	5
1.6 Significance of the research	5
1.7 Summary	6
CHAPTER TWO: LITERATURE REVIEW	7
2.1 Deepfake	7
2.2 Deepfake in Images	8
2.3 Deepfake in Videos	8
2.4 Deepfake in Audio	9
2.4.1 Text-to-Speech (TTS)	9
2.4.2 Voice Conversion (VC)	10
2.5 Audio Deepfake in Journalism	11
2.5.1 Effects of Audio Deepfake in Journalism	11
2.5.2 Authentication Methods	12
2.6 Audio Signal Processing Techniques for Deepfake Detection	13
2.6.1 Fast Fourier Transform (FFT)	13
2.6.2 The Short-Time Fourier Transform (STFT)	14

2.6.3	Spectral Centroid	14
2.6.4	Chroma Feature	15
2.6.5	Spectrogram	16
2.6.6	Mel-Spectrogram	16
2.6.7	Mel-Frequency Cepstral Coefficients (MFCC)	17
2.6.8	Constant-Q Cepstral Coefficients (CQCC)	19
2.6.9	Key audio features for audio Deepfake detection	19
2.7	Techniques used for Audio Deepfake Detection	21
2.7.1	Convolutional Neural Networks (CNN)	21
2.7.2	Recurrent Neural Networks (RNN)	22
2.8	Related Works	23
2.8.1	Unmasking the Truth: A Deep Learning Approach to Detecting Deepfake Audio through MFCC Features	24
2.8.2	Deepfake Audio Detection with Neural Networks using Audio Feature	26
2.8.4	Spoofing Detection of Fake Speech Using Deep Neural Network (DNN) Algorithm	30
2.8.5	Deepfake Audio Detection via MFCC Features Using Machine Learning	31
2.8.6	Discussion	37
2.9	Summary	38

CHAPTER 3: RESEARCH METHODOLOGY

3.1	Research Framework	39
3.2	Preliminary Study	40
3.3	Knowledge Acquisition	41
3.4	Data Acquisition	42
3.5	Data Pre-Processing	43
3.6	Model Design	Error! Bookmark not defined.
3.7	Model Development	43
3.8	Model Evaluation	44
3.9	Prototype Design	45
3.10	Prototype Development	45

3.11	Testing	46
3.12	Documentation	46
3.13	Summary	47

REFERENCES	101
-------------------	-----

APPENDICES

APPENDIX A	Error! Bookmark not defined.
------------	-------------------------------------

LIST OF TABLES

TABLE	PAGE
2.1 Key audio features for Audio Deepfake Detection	21
2.2 Performance of CNN-LSTM model	26
2.3 Parameters used in CNN model	28
2.4 Accuracy of CNN model with different audio features	28
2.5 Result for CNN using mel-spectrogram feature extraction	30
2.6 Fake-or-Real Dataset Description	33
2.7 Accuracy comparison for machine learning models	34
2.8 Accuracy comparison for noisy audio signals using machine learning models.	34
2.9 Observations on Previous Papers Used for Audio Deepfake Detection	37
3.1 Preliminary Study Phase	41
3.2 Knowledge Acquisition Phase	42
3.3 Data Acquisition Phase	43
3.4 Data Pre-Processing Phase	44
3.5 Model Design Phase	45
3.6 Model Development Phase	45
3.7 Model Evaluation Phase	46
3.8 Prototype Design Phase	47
3.9 Prototype Development Phase	47
3.10 Testing Phase	48
3.11 Documentation Phase	48

LIST OF FIGURES

FIGURE	PAGE
2.1 Deepfake of news anchor Kim Joo-Ha of the Korean television channel MBN	8
2.2 TTS process	10
2.3 typical voice conversion pipeline	11
2.4 Mel-Spectrogram	17
2.5 Steps to extract MFCC from an audio signal	18
2.6 Basic CNN Architecture	23
2.7 The basic architecture of RNN	24
2.8 Sequential CNN-LSTM	26
2.9 The Area Under the ROC Curve (AUC)	30
2.10 Performance comparison chart	32
2.11 VGG-16 model's training and validation accuracy and loss	35
3.1 Research Framework	40

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
GAN	Generative adversarial network
TTS	Text-To-Speech
VC	Voice Conversion
FFT	Fast Fourier Transform
STFT	Short-Time Fourier Transform
MFCC	Mel-Frequency Cepstral Coefficients
CQCC	Constant Q Cepstral coefficients
CNN	Convolutional Neural Networks
RNN	Recurrent Neural Networks
DNN	Deep Neural Networks

CHAPTER 1

INTRODUCTION

1.1 Research Background

The dominance of digital media characterises our era, where digital information can easily be created, communicated, and read globally. While this has expanded information availability, it also has made it increasingly difficult for citizens to verify and trust the vast amount of content available on the web. Recent advancements in artificial intelligence (AI) have had a significant impact across various domains, particularly in digital media, with critical implications. AI, particularly deep learning, is seen as a transformative technology due to its wide range of practical application, notably in the emergence and rise of deepfakes which has introduced new complexities (Karnouskos, 2020).

The emergence of Deepfake, a term blending “deep learning” and “fake” media, refers to digitally manipulated content where realistic representations are artificially created or modified to depict actions or statements that did not actually happen (Shoaib et al., 2023). Deepfakes initially garnered widespread attention in 2017 when a Reddit user known as ‘deepfakes’ utilised this technology to produce unsavoury manipulated content by swapping faces of celebrities using deep learning models (DL). These DL models are trained to generate very deceptively realistic replicas by merging and overlaying elements within the media (Mubarak et al., 2023). Deepfakes have attracted significant interest due to their ability to produce compelling and often indistinguishable fabricated content. In reality, deepfake stands out as a significant challenge facing modern society. The usage of deepfake is regularly used to steal popular Hollywood celebrities over pornographic picture films. Deepfake was also used to generate false information and rumours for politicians (Qais et al., 2022).

Deepfakes can be classified into four main categories: Text, Image, Video, and Audio. While most scientists are preoccupied with investigating deepfakes in video, the other types of deepfakes must also receive a wide range of attention due to the comprehensive advances in creating these types of deepfake (Shaaban et al., 2023). More specifically, audio deepfakes which are created using AI to generate manipulated audio content that closely resembles the voice of real person. This intricate process involves training machine learning models on extensive datasets of audio recordings from the target individual. The concern surrounding audio deep fakes lies in their potential to disseminate misinformation or tarnish the reputation of individuals and organisations (Altalahin et al., 2023). Furthermore, deepfake audio detection based on purely relying on audio data, has received less attention compared to image and video-based methods. This is because existing research often incorporates both audio and spatio-temporal information from videos to train deep learning models (Hamza et al. 2022).

A significant number of respondents to a global McAfee study conducted in 2023 reported suffering financial losses due to fraudulent activities facilitated by audio deepfakes. Moreover, the proliferation of audio deepfakes poses a threat to the security of financial customers, particularly impacting existing speech recognition systems utilised in the banking industry (McAfee, 2023).

1.2 Problem Statement

Deepfakes have become a potent tool for spreading misinformation and disinformation as they blur the line between reality and fabrication by creating highly convincing fake media, presenting a significant challenge to society's integrity (Al-Khazraji et al., 2023). With enough sample recordings of an individual it is possible to synthesise realistic audio of them making it hard to detect its manipulation (Argarwal et al., 2020). Malicious actors can use deepfakes to portray individuals saying or doing things they never actually did. In a notable accident from the year 2020, a bank manager had fallen victim to a 35\$ million transfer authorization as he believed he was speaking to a company director whose voice had been cloned by fraudsters (Kilinc, Kaledibi, 2023). Examples like these further proves that deepfakes in audio threatens the privacy, integrity and reliability of information made available in media (Khatri et al., 2023)

Due to the surge of social media and digitalization it has intensified the spread of fake news, challenging conventional definitions of news. False information disguised as fact floods online platforms daily. (Zhou & Zafarani, 2020) define false news as deliberately spreading incorrect materials that can be debunked by fact-checking. Research indicates that people across demographics struggle to identify false news. Deepfakes can spread fake news by malicious actors by manipulating publicly known individuals such as celebrities, politicians and major business leaders to spread false information or create chaos which can sow doubt, confuse public discourse, and undermine trust in institutions and leaders (Hartmann, Giles, 2020). An alarming example of this scenario is the circulation of a deepfake version of the Ukrainian president which lasted approximately one minute. The fabricated representation of the president appeared to instruct Ukrainian troops to cease fighting against Russia and surrender their weapons (Mubarak et al., 2023)

In recent years, with the rise of deepfake technology there has been an influx of deepfakes which is widely perceived as one of AI's most dangerous uses. This technology makes it possible to create totally imaginary situations towards an individual that they did never perform which has been detrimental towards democracy, society's security and people's privacy (Kaushal et al., 2022). To further proof towards the influx of deepfakes, according to Reuters "DeepMedia", a company who specialises in synthetic media detection tools, reported that there has been a significant increase of deepfakes online in 2023. Compared to 2022, voice deepfakes has increased eightfold in the year 2023. "DeepMedia" predicts that approximately half a million video and voice deepfakes will be circulated on global social media platforms by 2023. This has the potential to significantly impact various aspects of society (Mubarak et al., 2023). Currently measures are taken to solve these problems from researchers from all around the world by developing effective deepfake detection techniques (Kaushal et al., 2022). This project will help further the advancement of deepfake detection by finding a suitable model for detecting deepfaked audio and developing a web based prototype to detect said audio whether it is deepfaked or not. This development allows users to upload or input media files to verify the authenticity of the audio content.

1.3 Research Questions

The research questions for this project as follow:

- i. Is there any key audio feature that can be effectively identified and utilized to differentiate deepfaked audio recordings from genuine recordings?
- ii. What model is suitable in identifying whether an audio is deepfaked?
- iii. How to develop an audio deepfake detection system?

1.4 Research Objectives

The research objectives of this project consists of:

- i. To identify key audio features that effectively differentiate deepfaked audio recordings from genuine recordings
- ii. To evaluate deep learning models to detect deepfaked audio
- iii. To develop a website based prototype for detecting deepfake audio

1.5 Research Scope

This study will focus primarily on journalism and on how it will help in combating misinformation. This is done by a training dataset found on The automatic speaker verification spoofing and countermeasures (ASVspoof). ASVspoof 2021 is the 4th in a series of bi-annual, competitive challenges where the goal is to develop countermeasures capable of discriminating between bona fide and spoofed or deepfake speech. The findings will then be implemented into a website prototype, providing journalists and the public with a platform to verify audio content and combat the spread of misinformation

1.6 Significance of the research

Deepfake audio can be used to spread false information, manipulate public opinion, and undermine democratic processes. By detecting and flagging deepfake audio, the study contributes to the fight against misinformation, enabling journalists and media organisations to provide accurate and reliable information to the public. By enhancing the ability to detect deepfake audio, the research will help to achieve the 16th Sustainable Development Goal (SDG) as the research contributes to promoting transparency, accountability, and integrity in media institutions, which are essential for building strong and inclusive societies.

1.7 Summary

In summary, this chapter has delved into the research background to provide a deeper insight into the subject area. It has examined the problem statements to explain the challenges that arise and the objectives this project aims to tackle, as well as the proposed project intended to address these issues. The proposed project aims to accomplish the intended goals outlined. The scope is outlined to illustrate the project's boundaries and limitations. The significance of this project is to elucidate the benefits and contributions it brings forth.

CHAPTER 2

LITERATURE REVIEW

2.1 Deepfake

Under the guise of computer vision and deep learning technology, an innovative technique has emerged that enables the creation of highly realistic yet fake videos, images, and even manipulated voices. This technology, widely known as Deepfake Technology, which derives their name from their uses of advanced deep learning technology to generate fraudulent content in various forms, including videos, images, texts and voices. GANs are commonly used to produce images and videos. Nowadays Deepfake technology has become more popular due to its ease and cheap availability. First ever success of Deepfake technique was the creation of FakeApp which is used to swap a person's face with another person (Mahmud & Sharmin, 2020).

GANs are powerful deep learning algorithms that consist of two neural networks called a generator and a discriminator. The generator creates fake content, while the discriminator evaluates its authenticity. Better results can be obtained with the discriminator net since it rejects certain subpar instances even when the autoencoder net and generator net have the same appearance. Therefore, it is assumed by GAN's deep-fake creation method that the generator's objective is to outsmart the discriminator. The interaction between these networks results in highly realistic deepfakes. GANs are challenging to train but are effective in producing content that closely resembles real videos, making it difficult for humans to distinguish the fakes. Open-source projects like Faceswap-GAN employ this method (Garg & Gill, 2023).

2.2 Deepfake in Images

Deep learning, particularly through Artificial Neural Networks (ANNs), is extensively used in computing fields like visual identification, cybersecurity, and text analytics. GANs are particularly effective in synthesizing near-impossible-to-detect fake images. This is because deep networks have demonstrated great effectiveness in various regression and classification challenges when properly trained. Despite requiring substantial training data and time, GANs have proven to be powerful tools for generating realistic images. However, their application to videos is less common due to challenges in maintaining temporal consistency (Kaushal et al., 2022).

2.3 Deepfake in Videos

Creating deepfake videos involves using GANs, where a generator creates the video and a discriminator assesses its authenticity. The discriminator's feedback helps refine the generator, resulting in videos that are hard to distinguish from real ones (Das et al., 2022). A famous example of said video deepfake is the South Korean television channel MBN using a deepfake of its news anchor Kim Joo-Ha which can be seen in figure 2.1. The broadcaster informed viewers beforehand about the deepfake, which was created by DeepBrain AI. MBN plans to continue using deepfakes for breaking news reports (Mekhail Mustak et al., 2023).



Figure 2.1 Deepfake of news anchor Kim Joo-Ha of the Korean television channel MBN.

(*Source:* Mekhail Mustak et al., 2023)

2.4 Deepfake in Audio

AI has contributed to major advances in many areas of digital technology, with audio signal processing being one of the primary beneficiaries. From voice assistants to audio transcription services, AI has significantly transformed the audio landscape. Speech synthesis systems have been extensively employed in various applications such as audiobooks, despite their increasing misuse in generating DeepFake audios (Alberic et al., 2023). The advancements in TTS and VC technologies have significantly enhanced the ease and quality of creating audio deepfakes. These deepfakes, often referred to as "spoofs," can be used for various purposes, including malicious activities like identity fraud and spreading misinformation. The complexity of these technologies underscores the importance of developing advanced detection and monitoring systems to distinguish genuine audio from synthetic fabrications.

2.4.1 Text-to-Speech (TTS)

Text-to-Speech (TTS) systems convert written text into spoken words by analyzing the text and generating speech that aligns with the input using linguistic rules. TTS can be used for various purposes, such as reading text aloud or serving as personal AI assistants. Additionally, TTS systems can offer different accents and voices beyond pre-recorded human voices. Companies like Lyrebird utilize deep learning models to generate vast amounts of speech rapidly. However, TTS systems heavily depend on the quality of the speech corpus, which is expensive to create. They also struggle with recognizing punctuation and handling homographs which is when two words that are spelled the same but have different meanings (Khanjani et al., 2021).

The TTS process consists of three main modules: a text analysis module, an acoustic module, and a vocoder. The text analysis module processes the incoming text and converts it into linguistic characteristics. The acoustic module

then extracts parameters of the target speaker based on the linguistic features generated by the text analysis module. Finally, the vocoder creates speech waveforms from these acoustic parameters, producing the final synthetic audio in waveform format (Almutairi & Hebah Elgibreen, 2023)

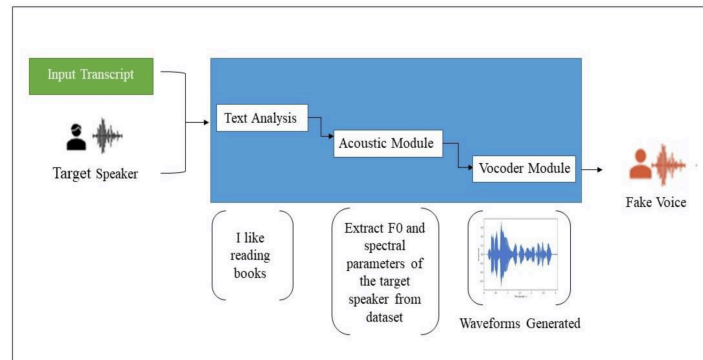


Figure 2.2 TTS process

(Source: Saranga Kingkor Mahanta et al., 2021)

2.4.2 Voice Conversion (VC)

Unlike Text-to-speech, Voice Conversion (VC), also known as speech-to-speech, takes a speech signal from one speaker (the source) and modifies it to sound like another speaker (the target), preserving the linguistic content and other aspects of the target speaker's identity. This technique can be considered a form of impersonation and has been made more accessible with new technologies. For example, the company Overdub can create an impression of any voice with just a one-minute audio sample. GANs are also employed for voice impersonation. In the entertainment industry, VC is used for dubbing foreign films or TV shows with the original actors' voices. Additionally, it can help individuals with speech disabilities communicate effectively by generating speech that matches their authentic voice (Tomasz Walczyna & Piotrowski, 2023) Figure 2.3 below shows a typical voice conversion pipeline

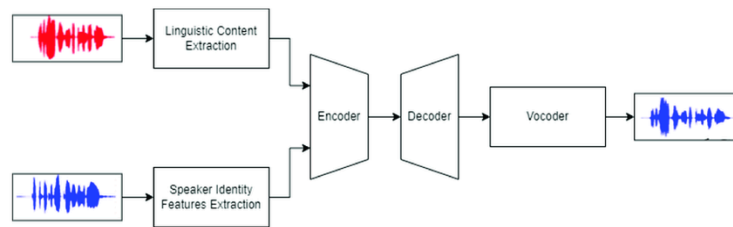


Figure 2.3 typical voice conversion pipeline
 (Source: Tomasz Walczyna & Piotrowski, 2023)

2.5 Audio Deepfake in Journalism

Technically, Journalism's role as the fourth estate is predicated on the ability to provide accurate, reliable information. Deepfakes pose existential challenges to this role. Journalists are forced to contend with the additional burden of verifying content authenticity while the public grows increasingly skeptical of media reports (Wahl-Jorgensen & Carlson, 2021). This skepticism can lead to a 'cry wolf' scenario, where even legitimate news contents are doubted, contributing to a disconcerting post-truth era where facts are fungible, and the truth is subjective (Shoaib et al., 2023). One notable incident involving audio deepfakes is the altered video of US Speaker of the House Nancy Pelosi, which circulated on social media and highlighted the technology's potential for harm

2.5.1 Effects of Audio Deepfake in Journalism

Deepfakes, particularly audio deepfakes, pose significant challenges to journalism by threatening the credibility and authenticity of news. The potential of deepfakes to fabricate audio, such as fake interviews or false news reports, creates a scenario where misinformation can be easily and convincingly spread. This technology, which remains in its nascent stages, has journalists more focused on its future disruptive potential rather than current impacts. Deepfakes are believed to exacerbate the existing issue of fake news, making it even harder for the public to trust audiovisual media. This concern is rooted in the historical context of how new technologies, such as the Great Moon Hoax of 1835 and Orson Welles's War of the Worlds broadcast, have historically manipulated

public perception. The fear is that deepfakes, by presenting highly believable yet false content, could lead to widespread confusion and mistrust in media sources. These challenges force journalists to confront and redefine the line between credible news and deceptive hoaxes, which jeopardize their role as reliable sources of accurate information (Creech & Roessner, 2018)

2.5.2 Authentication Methods

Journalists employ various methods to authenticate media to combat the rise of deepfakes. Fact-checking, responsible sourcing, and gatekeeping are traditional methods that remain central. In addition, technological tools and collaborations with AI experts help detect anomalies indicative of deepfakes. The role of digital forensics is becoming increasingly important, involving sophisticated analysis techniques to verify the authenticity of audio and visual content. Social media platforms also play a crucial role by moderating and restricting the spread of suspected deepfake content (Karnouskos, 2020).

To address the challenges posed by deepfakes, journalists and media organizations are enhancing their detection and mitigation strategies. This includes the use of advanced software that can identify the subtle imperfections and inconsistencies often present in deepfakes. Media literacy campaigns are also crucial, educating the public on the potential for deepfakes and encouraging skepticism towards suspicious content. Journalists advocate for their role in upholding truth and emphasize the importance of trusted, professional journalism in an era where deepfakes could undermine democratic processes and public trust. The journalistic discourse around deepfakes often stresses the need for vigilance and the reassertion of journalistic authority as a defense against the erosion of trust in media (Karnouskos, 2020).

2.6 Audio Signal Processing Techniques for Deepfake Detection

Audio feature extraction involves analyzing and extracting meaningful information from audio signals to create a concise and informative representation that computers can process. These features are typically designed within specific tasks and fields, such as audio segmentation, automatic speech recognition, music information retrieval, and general sound recognition. Interestingly, features developed for one task often find utility in other domains over time. For instance, Mel-frequency cepstral coefficients (MFCCs) were originally developed for automatic speech recognition but have since been adapted for use in music information retrieval and environmental sound analysis. This versatility suggests that audio features can transcend their initial applications and be valuable across different domains (Dalibor Mitrović et al., 2010).

Deepfake audio signals often share similar feature sets with the original signals, making them difficult to distinguish, even with advanced deep learning techniques. Consequently, the features extracted from these signals significantly impact the model's predictive power and accuracy. Analyzing audio signals in the frequency domain has been observed to yield features useful for detecting and classifying deepfake audios, which can sometimes deceive human listeners under certain conditions (Hamza et al., 2022).

2.6.1 Fast Fourier Transform (FFT)

The Fast Fourier Transform (FFT) is an algorithm designed to efficiently compute the Discrete Fourier Transform (DFT) and its inverse. By breaking down a time-domain signal into its constituent frequency components, the FFT reveals the signal's frequency spectrum. This transformation is crucial for analysing the frequency characteristics of signals in various domains, including audio processing. The FFT is widely used to convert signals from the time domain to the frequency domain, enabling the identification of periodic

components and the analysis of signal properties across different frequencies (Qais et al., 2022).

2.6.2 The Short-Time Fourier Transform (STFT)

The Short-Time Fourier Transform (STFT) extends the traditional Fourier Transform by analysing a signal's frequency content over short, overlapping time segments. This approach allows for the examination of how the signal's frequency spectrum changes over time, providing a time-frequency representation. The STFT involves dividing the signal into segments of equal length, applying a window function to each segment, and then computing the Fourier transform for each windowed segment. The result is a matrix of complex numbers representing the signal's frequency content at different times. This method is particularly useful for non-stationary signals whose frequency characteristics vary over time. The STFT forms the basis for many audio processing techniques and is used to compute various features, such as the spectral centroid and spectral slope (R.L.M.A.P.C. Wijethunga et al., 2020).

2.6.3 Spectral Centroid

The spectral centroid represents the centre of mass of a signal's magnitude spectrum, effectively pinpointing the "centre of gravity" of the spectrum. It is defined as the frequency at which the energy distribution of the spectrum is centred. This metric is associated with the perceptual attribute of the brightness of a sound; a higher spectral centroid usually indicates a brighter, more energetic tone, whereas a lower centroid corresponds to a duller, more subdued sound. The concept extends to the spectral spread, which measures the dispersion around the spectral centroid, and spectral bandwidth, which reflects how energy is distributed across the frequency range. Mathematically, the spectral centroid is the weighted mean of the frequencies present in the signal, with their magnitudes

as the weights, providing a quantitative measure of the tonal quality of audio (Qais et al., 2022)..

2.6.4 Chroma Feature

The chroma feature is a prominent audio melody descriptor extensively used in the field of music information retrieval. It is crafted based on the principles of Twelve Tone Equal Temperament, which divides an octave into 12 equal parts, each corresponding to a semitone. In musical contexts, notes separated by exactly one octave are perceived as highly similar. Therefore, understanding the distribution of chroma—without needing the absolute frequency (or the specific octave)—can provide valuable musical insights about the audio. This feature can also uncover perceived musical similarities that might not be immediately evident from the original spectra. Typically, the chroma feature is represented as a 12-dimensional vector, $v = [V(1), V(2), V(3), \dots, V(12)]$. Each element of this vector corresponds to one of the 12 pitch classes: {C, C#, D, D#, E, F, F#, G, G#, A, A#, B}. This representation reflects the local energy distribution of the audio signal at the semitone levels represented by these pitch names. By capturing the energy at each pitch class, chroma features provide a compact and musically meaningful representation of the audio, facilitating various tasks in music information retrieval such as chord recognition, key detection, and music similarity analysis. This detailed chroma representation enables the detection of harmonic structures and relationships within the audio signal, making it a powerful tool for musical analysis and processing (Shi et al., 2019)

2.6.5 Spectrogram

A spectrogram serves as a visual representation of an audio signal's spectrum as it varies over time, offering a detailed view of how the signal's frequency content evolves. This visualisation is achieved by applying the Short-Time Fourier Transform (STFT), which involves segmenting the signal into overlapping frames and applying the Fast Fourier Transform (FFT) to each frame. The result is a time-frequency representation where time is plotted on the horizontal axis, frequency on the vertical axis, and the amplitude or power of each frequency component is indicated by colour intensity. This approach allows for the detailed analysis of signal characteristics, including changes in frequency and amplitude over time. Spectrograms are particularly useful for identifying patterns and features in audio data, such as phonemes in speech or notes in music (Zeinali et al., 2019).

2.6.6 Mel-Spectrogram

The Mel-spectrogram is not the same as a regular spectrogram since it is based on human auditory perception and modifies a standard spectrogram by changing its frequency axis to the Mel scale. Pitch is perceived logarithmically by humans, variations in low frequencies are easier to hear than that of high frequencies. As a result, sound is represented more perceptually meaningfully in a Mel spectrogram compared to the standard spectrogram. Using the Mel scale formula, a spectrogram's frequency axis is compressed to produce a representation that is more in line with human hearing. equivalent distances on the Mel scale correlate to equivalent perceptual differences in pitch because the Mel scale spaces frequencies logarithmically. The Mel spectrogram is a useful tool for speech and audio processing applications because of this transformation, which highlights perceptually significant aspects of the sound. Mel is related to hertz by the equation below.

$$m = 2595 * \log_{10}(1 + f/700)$$

The mel spectrogram is widely used in modern speech synthesis models. Indeed, in models such as tacotron, deepvoice, multispeech or fastspeech, the acoustic parameter prediction models output mel-spectrograms that the vocoder converts into a waveform, even though the last two models are able to synthesise waveforms directly.(Alberic et al., 2023)

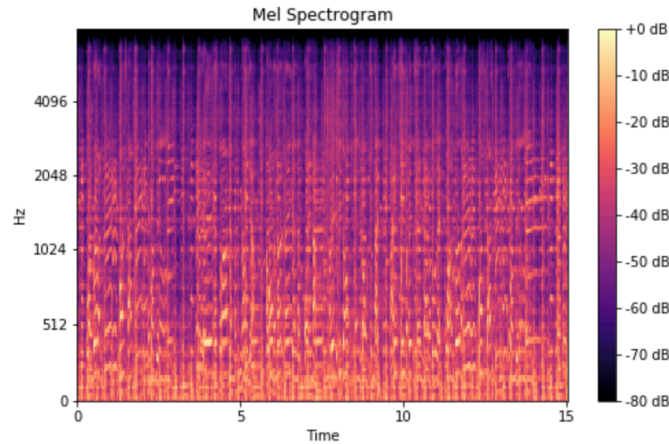


Figure 2.4 Mel-Spectrogram
(Source: Guo et al., 2023)

2.6.7 Mel-Frequency Cepstral Coefficients (MFCC)

Mel-Frequency Cepstral Coefficients (MFCCs) are derived from a signal's cepstrum, which encapsulates the rate of change in different spectral bands. The cepstrum itself is obtained by taking the logarithm of the signal's power spectrum and then applying the inverse Fourier transform. The resulting coefficients, known as Mel-Frequency Cepstral Coefficients or MFCC in short, represent the signal's spectral envelope and are highly effective for characterising the timbre of audio signals. MFCCs capture important perceptual aspects of sound, such as formants and pitch, making them indispensable in speech and audio recognition systems. The coefficients are calculated by applying a filter bank spaced according to the Mel scale to the power spectrum of the signal, followed by a discrete cosine transform. This process produces a set of coefficients that

succinctly describe the shape of the spectrum, facilitating tasks like speech recognition and speaker identification (Qais et al., 2022).

MFCC features are widely used in speech and audio processing, as they provide a compact representation of the spectral shape of the audio signal. They are derived from the Mel scale, a perceptually motivated frequency measure that approximates the response of the human auditory system to sound. MFCCs can capture the temporal properties of sound, which can be useful in detecting fake sounds, as different sound generation methods may have distinct sound signatures (Altalihin et al., 2023). According to (Shashidhar et al. 2012), the following steps are performed to obtain MFCC from a speech signal:

- Fourier transform of a given speech segment
- Calculate the powers of the above spectrum within the triangular overlapping windows placed according to mel scale
- Take logs of the power at each mel frequencies
- Calculate the DCT (Discrete cosine transform) of the list of mel log powers
- The amplitude of resulting spectrum gives MFCC's

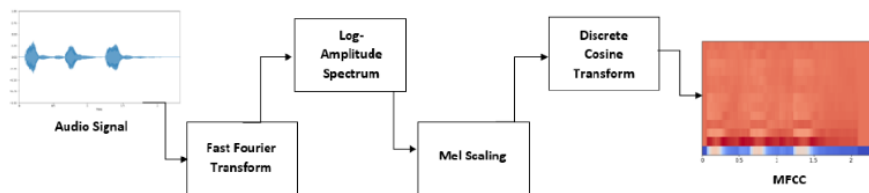


Figure 2.5 Steps to extract MFCC from an audio signal

(Source: Saranga Mahanta et al., 2021)

2.6.8 Constant-Q Cepstral Coefficients (CQCC)

Constant Q Cepstral coefficients (CQCCs) utilize a more perceptually motivated time-frequency analysis method called constant Q transform (CQT) to extract features, which greatly improves the accuracy and is considered as a state-of-the-art feature for detecting spoofed speeches. The constant Q factor, which is the ratio of center frequency to bandwidth is maintained through geometrically spaced bins. This ensures higher frequency resolution at lower frequencies and higher temporal resolution at higher frequencies. The block diagram for the CQCC feature extraction scheme is depicted in the figure.

In the CQCC feature extraction process, the constant Q transform(CQT) of the input speech signal is first computed, converting the signal from the time domain to the frequency domain. The power spectrum is then used to derive the octave power spectrum, followed by taking the logarithm. Uniform sampling converts the logarithmic octave power spectrum into a linear logarithm power spectrum. Finally, the decorrelated coefficients are obtained using the Discrete Cosine Transform (DCT) (Chaudhari & Shedge, 2022). In other words, CQCC is retrieved based on CQT, while MFCC is extracted from mel-scale.

2.6.9 Key audio features for audio Deepfake detection

Based on these audio features that are selected, the one that stands out for the specific audio deepfake detection tasks are Mel-Frequency Cepstral Coefficients (MFCC), Constant-Q Cepstral Coefficients (CQCC), Mel-spectrogram, and Short-Time Fourier Transform (STFT). These features are chosen based on their ability to effectively capture and represent the critical characteristics of audio signals necessary for distinguishing genuine recordings from deepfaked ones.

Mel-Frequency Cepstral Coefficients (MFCC) are chosen due to their ability to align well with human auditory perception and the fact that MFCC are widely

used in speech and audio recognition systems, making them highly suitable for detecting deepfake audio. They distinctly describe the shape of the spectrum, facilitating tasks such as speech recognition and speaker identification, which are essential for identifying inconsistencies introduced by deepfake generation methods. Constant-Q Cepstral Coefficients (CQCC) come next, as it offers a more detailed time-frequency analysis method that improves accuracy in detecting spoofed speeches. By maintaining a constant Q factor, CQCCs provide high resolution at lower frequencies and accurate time-frequency representation. Their state-of-the-art performance in detecting spoofed audio makes them a strong choice. The Mel-spectrogram, which modifies the standard spectrogram by aligning its frequency axis to the Mel scale, provides a perceptually meaningful representation of sound, highlighting important audio aspects related to human hearing, making it highly effective for speech and audio processing. Finally, the Short-Time Fourier Transform (STFT) extends the traditional Fourier Transform by providing a time-frequency representation, making it suitable for analyzing non-stationary signals whose frequency characteristics vary over time. This method captures how the signal's frequency spectrum changes, which is vital for detecting the transient anomalies often present in deepfake audio.

Table 2.1 Key audio feature for Audio Deepfake Detection

Audio Features	Reason
MFCC	- Aligns well with human auditory perception, making it effective for tasks related to speech recognition and speaker identification. - Widely used in speech and audio recognition systems, facilitating the detection of inconsistencies from deepfake methods.

2.7 Techniques used for Audio Deepfake Detection

In the realm of audio deepfake detection, advanced neural network architectures such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) play pivotal roles. CNNs are particularly effective in extracting intricate features from audio signals by treating them similarly to image data, leveraging their hierarchical structure to recognize complex patterns and topological properties. On the other hand, RNNs are adept at modeling the temporal dependencies inherent in audio data, making them suitable for analyzing time-series information like speech.

2.7.1 Convolutional Neural Networks (CNN)

Convolutional Neural Networks (CNNs) have a hierarchical network structure capable of both feed-forward and back-propagation operations. In feed-forward propagation, input data moves through multiple layers, each applying an activation function, ultimately producing an output in the final layer. Conversely, back-propagation calculates the error between the generated output and the actual sample, adjusting the network's parameters to minimise this error.

The performance of CNNs in image processing has led to a significant increase in their usage. Initially, image retrieval systems relied on text-based methods, where images were manually analysed and indexed. As image databases grew, this process became increasingly burdensome. CNNs, being feed-forward neural networks, excel at extracting topological properties from images, enabling them to recognize highly variable patterns. This capability has made Deep CNNs popular among researchers for various image processing tasks (Pukale, 2017).

A CNN is composed of neurons arranged in an acyclic graph with multiple hidden layers. Each neuron is connected to a subset of neurons from the previous layer. Like other artificial neural networks, CNNs consist of neurons with learnable weights and biases. Neurons receive numerous inputs, compute a weighted sum, pass it through an activation function, and produce the desired output. (Arora et al., 2020) The basic architecture of a CNN (as can be seen in figure 2.6) includes four primary layers: The Convolution Layer, Pooling Layer, Fully Connected Layer, and Loss Layer (Neena Aloysius & M. Geetha, 2017)

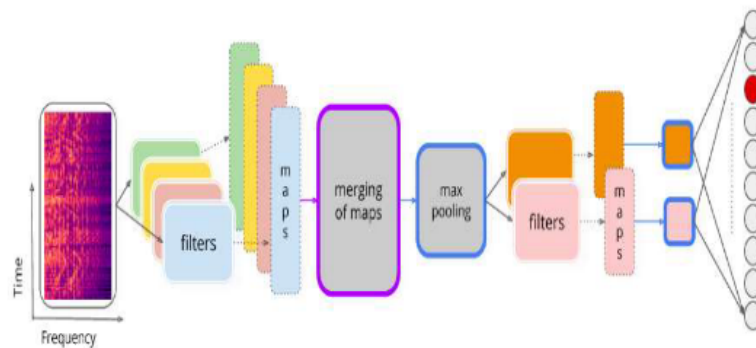


Figure 2.6 Basic CNN Architecture
(Source: Qais et al., 2022)

2.7.2 Recurrent Neural Networks (RNN)

Recurrent Neural Networks (RNNs) are advanced tools for modelling time-series data such as speech signals, language texts, video frames, body sensor data, and weather information. Unlike Convolutional Neural Networks (CNNs), which excel at capturing spatial information, RNNs are specifically designed to handle the temporal dimension of data. As a result, RNNs have become powerful solutions for a variety of complex tasks (Lalapura et al., 2022).

A Recurrent Neural Network (RNN) is a type of neural network where the output from the previous step is used as input for the next step. Unlike traditional neural networks where all inputs and outputs are treated independently, RNNs are designed to handle sequential data where the context from previous inputs is

important. For example, when predicting the next word in a sentence, the preceding words provide necessary context. RNNs address this by incorporating a hidden layer that retains information about the sequence, effectively giving the network a form of memory. This memory enables RNNs to maintain information about previous computations, using the same parameters for each input to produce consistent results. This approach simplifies parameter complexity compared to other neural network models. However, when there is a significant gap between relevant pieces of input data, standard RNNs struggle. To address this issue, Hochreiter and Schmidhuber introduced Long Short-Term Memory (LSTM) networks in 1997, which are capable of handling long-term dependencies. LSTM networks have become central to deep learning due to their ability to achieve impressive results in various applications involving RNNs. In RNNs, the hidden layers, also referred to as recurrent layers, consist of cells whose states are influenced by both previous states and current input through feedback connections (Malik et al., 2022).

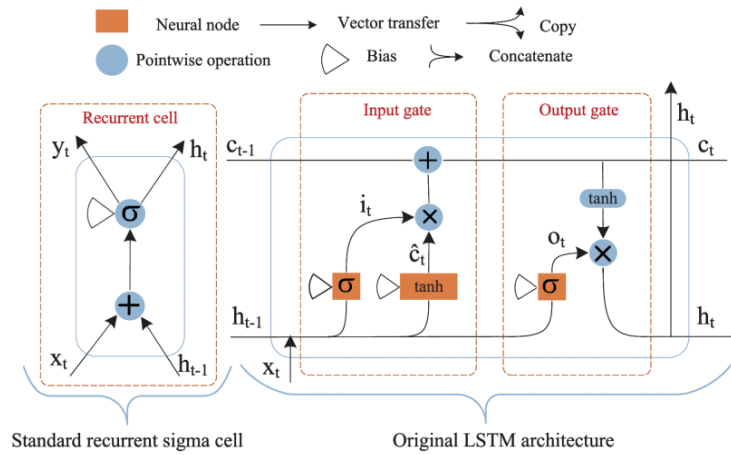


Figure 2.7 The basic architecture of RNN

(Source: Malik et al., 2022)

2.8 Related Works

Related works serve as a crucial process in understanding the current landscape, including the methodologies, datasets, and performance metrics used that have been previously explored by other researchers on audio deepfake detection.

2.8.1 Unmasking the Truth: A Deep Learning Approach to Detecting Deepfake Audio through MFCC Features

The paper "Unmasking the Truth: A Deep Learning Approach to Detecting Deepfake Audio through MFCC Features" by Altalahin et al. (2023) explores techniques for identifying deepfake audio using deep learning. The primary technique employed in this study is the use of Melted Frequency Cepstral Coefficients (MFCCs) for feature extraction, which are known for capturing the spectral properties of audio signals. This method effectively reduces the dimensionality of audio data while retaining its essential characteristics, making it suitable for detecting anomalies introduced by deepfake generation processes.

The dataset utilized in this research is the ASVspoof 2019 dataset, which is a standard reference for evaluating anti-spoofing systems. The ASVspoof 2019 database includes various forms of spoofing attacks, categorized into logical access (LA) and physical access (PA) scenarios. The dataset is further divided into training, development, and evaluation subsets, each containing distinct speakers and spoofing methods to ensure comprehensive assessment.

For model architecture, the study leverages a hybrid Convolutional Neural Network (CNN) combined with Long Short-Term Memory (LSTM) networks, referred to as CNN-LSTM. This architecture is adept at handling sequential data like audio signals. The CNN layers are responsible for extracting local patterns

from the input features, while the LSTM layers capture temporal dependencies, thereby enhancing the model's ability to detect subtle differences between genuine and fake audio.

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 185, 128)	512
max_pooling1d (MaxPooling1D)	(None, 92, 128)	0
dropout (Dropout)	(None, 92, 128)	0
conv1d_1 (Conv1D)	(None, 90, 128)	49280
max_pooling1d_1 (MaxPooling1D)	(None, 45, 128)	0
dropout_1 (Dropout)	(None, 45, 128)	0
separable_conv1d (Separable Conv1D)	(None, 43, 128)	16896
max_pooling1d_2 (MaxPooling1D)	(None, 21, 128)	0
lstm (LSTM)	(None, 21, 256)	394240
lstm_1 (LSTM)	(None, 32)	36992
dropout_2 (Dropout)	(None, 32)	0
flatten (Flatten)	(None, 32)	0
dense (Dense)	(None, 128)	4224
dropout_3 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 2)	258

Figure 2.8 Sequential CNN-LSTM

(Source: Altalahin et al., 2023)

The experimental results highlight the effectiveness of the proposed CNN-LSTM model, achieving an accuracy of up to 88% in detecting deepfake audio. Other performance measures have also been used as shown in Table 2.2, the table represents the analysis of the results of the experiment to determine the accuracy of the data and evaluate the strengths and weaknesses of this algorithm. This performance demonstrates the potential of combining MFCC features with CNN-LSTM architectures for addressing the challenges posed by increasingly sophisticated deepfake technologies. The study concludes that this approach is promising and suggests further research to continue improving the detection capabilities for more robust defense against deepfake audio.

Table 2.2 Performance of CNN-LSTM model

	Precision	Recall	F1-score	Support
0	0.83	0.96	0.89	313
1	0.95	0.79	0.86	287
Accuracy	-	-	0.88	600
Macro avg	0.89	0.87	0.87	600
Weighted avg	0.89	0.88	0.88	600

(Source: Altalahin et al., 2023)

2.8.2 Deepfake Audio Detection with Neural Networks using Audio Feature

Qais et al. (2022) proposed a speech spoofing detection system based on CNN using different audio features to classify between human speech and synthetic voice. In the research, the primary model that is used is CNN which then will be fed with audio features. These features include spectrograms, MFCC, STFT and FFT.

The dataset used for this research is called the ASVspoof 2017 dataset, which is specifically designed to explore a variety of activities related to verification, from detection of deception to counter measures play the attack again. Due to this reason the researchers specifically utilized the portion of the dataset relevant to verification tasks, which is described as logical access in the database. The data was then divided into training and testing sets in an 80:20 ratio. As the research works with audio data, which is organically larger in size compared to numerical data. To address this issue, the researchers went ahead and converted the audio files into images. The model with its parameters as seen in table 2.3 is then implemented into python and it is seen that MFCC features with Relu as an activation function has an accuracy of 75.97% which outperformed every other feature. Spectrogram has shown to perform with the least accuracy with 68.9% and concatenated data which combines all of the features performed the best with the accuracy of that being 80.47%. Table 2.4 shows the accuracy table for CNN with ReLu as an activation function.

Table 2.3 parameters used in CNN model

Parameters	Description
Epochs	30
Steps per Epoch	8
MaxPooling2D output shape	(nan, 99, 99, 16)
Convolutional Layer Filters	16
Audio Features Input Dimension	200x200x3
Max Pooling Layer Output Dimension	100x100x3

(Source: Qais et al., 2022)

Table 2.4 Accuracy of CNN model with different audio features

MODEL	ACCURACY
df_MFCC_relu	75.97%
df_STFT_relu	73.34%
df_FFT_relu	71.48%
df_SPEC_relu	70.86%
df_concatated_relu	80.47%

(Source: Qais et al., 2022)

The authors acknowledged that the dataset is a few years old, and using a more up-to-date dataset would be beneficial since spoofing technology evolves more rapidly than detection technology. Additionally, the authors proposed that Recurrent Neural Networks (RNNs) may be preferable to Convolutional Neural

Networks (CNNs) because RNNs provide feedback by feeding the results back into the network.

2.8.3 Audio Deepfake Detection Using Deep Learning

The paper done by R. Anagha et al. (2023) employs the model architecture of CNN. The paper also makes use of mel-spectrograms as the extracted audio feature which are then either padded or truncated to a fixed width. The model starts with an input layer that takes Mel spectrograms. This is followed by two convolutional layers with 32 and 64 filters respectively, each followed by ReLU activation functions. To reduce spatial dimensions, max-pooling layers with a 2x2 pool size are applied after each convolutional layer. A flattening layer then converts the pooled feature maps into a single vector, which is processed by a fully connected dense layer using ReLU activation. To prevent overfitting, a dropout layer with a 0.5 rate is included before the final dense layer with softmax activation for binary classification.

The training process involved splitting the dataset into 80% for training and 20% for validation. The Adam optimizer was utilized along with categorical cross-entropy loss to adjust learning rates dynamically based on prior gradient information, optimizing the convergence process. Dropout was applied during training to ensure the model does not rely too heavily on specific pathways and can adapt to diverse data patterns.

The research employs the ASVspoof 2019 dataset, which includes both real (bona fide) and fake (spoofed) audio recordings. Real samples are recordings of actual human speech, providing authentic speaker data to establish a baseline for genuine audio. In contrast, fake samples include altered recordings like replay

attacks and voice synthesis designed to deceive speaker verification systems. The dataset features a diverse range of recording conditions, including various environments and levels of background noise, to better simulate real-world scenarios.

The model's performance was evaluated using several metrics. The accuracy of the model was reported at 85%, indicating the overall correctness of its predictions. The Area Under the ROC Curve (AUC) as can be seen in figure 2.9 was 0.87, reflecting the model's capability to distinguish between real and fake audio. The average precision, which measures the model's precision across various classification thresholds, was reported as 0.90, highlighting the model's effectiveness in precision-recall trade-offs.

Table 2.5 Result for CNN using mel-spectrogram feature extraction

Serial Num	Metric Nameã	Result
1	Accuracy	85%
2	Area Under the ROC Curve (AOC)	0.87
3	Average Precesion	0.90

(Source: R. Anagha et al., 2023)

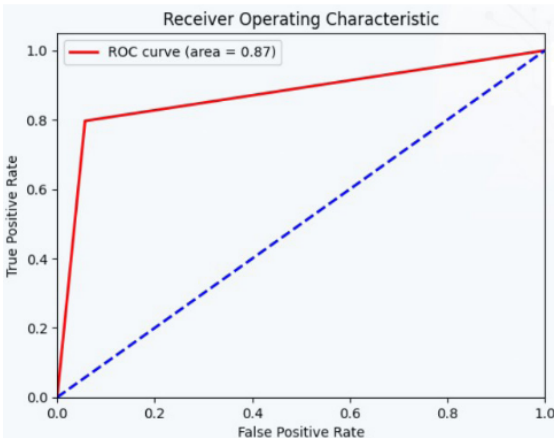


Figure 2.9 The Area Under the ROC Curve (AUC)

(Source: R. Anagha et al., 2023)

To conclude, the study presents an effective method for detecting audio deepfakes using CNNs. By leveraging Mel spectrograms for feature extraction and employing a CNN architecture, the model demonstrates high accuracy and robustness in distinguishing between real and fake audio. The results suggest that CNNs are well-suited for this task, contributing significantly to the field of audio forensics and enhancing the security and integrity of audio data.

2.8.4 Spoofing Detection of Fake Speech Using Deep Neural Network Algorithm

This paper done by Kristiawan Nugroho & Edy Winarno (2022) focuses on detecting fake speech using a combination of machine learning and deep learning techniques. In this research, three machine learning methods were used in overcoming various problems in data mining, namely Naive Bayes (NB), K-Nearest Neighbour (KNN) and Logistic Regression (LR). Additionally, to provide a solution towards the limitations of the machine learning method in this study, the Deep Neural Network (DNN) approach is highlighted for its capability to manage large datasets and uncover complex patterns.

The study uses a dataset composed of 10,000 speech samples, each labeled as either REAL or FAKE. The REAL samples represent genuine speech, while the FAKE samples consist of spoofed speech. Preprocessing of the dataset involves cleaning the audio data to reduce background noise. Feature extraction is performed using the Mel-Frequency Cepstral Coefficient (MFCC) method, which is effective in capturing the characteristics of the speech signals. This preprocessing and feature extraction ensure that the data fed into the classification models is of high quality and relevant for the task.

The performance of the various models is evaluated using metrics such as accuracy, precision, recall, and F1 score. As can be seen in figure 2.10 the Naive Bayes model achieves an accuracy of 87.5%, precision of 76.5%, recall of

87.5%, and an F1 score of 81.6%. The K-Nearest Neighbor model performs slightly better with an accuracy of 89.1%, precision of 94.2%, recall of 89.1%, and an F1 score of 90.4%. Logistic Regression matches the Naive Bayes model in terms of accuracy and recall, both at 87.5%, but shares the same precision and F1 score as well. The Deep Neural Network model significantly outperforms the others, achieving an accuracy of 96.5%, precision of 97.3%, recall of 96.5%, and an F1 score of 96.7%. These results clearly indicate that the DNN method excels in detecting fake speech, demonstrating superior performance across all evaluated metrics compared to the other machine learning techniques.

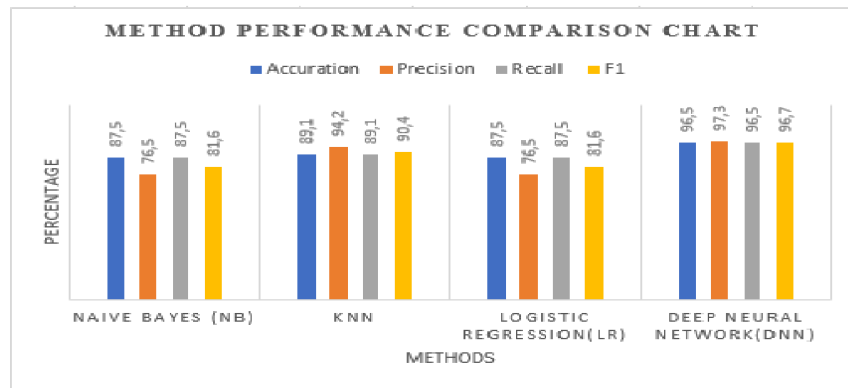


Figure 2.10 Performance comparison chart

(Source: Kristiawan Nugroho, & Edy Winarno., 2022).

2.8.5 Deepfake Audio Detection via MFCC Features Using Machine Learning

In the paper done by Hamza et al. (2022) titled “Deepfake audio detection via MFCC features using machine learning” propose a transfer learning-based approach to detect deepfake audio which employs a variety of machine learning and deep learning techniques. As the title suggests the primary technique that is involved in this study includes the feature extraction of Mel-Frequency Cepstral Coefficients (MFCCs) and short-time Fourier transform (STFT). In addition to MFCC, features such as Spectral (Roll-off point, centroid, contrast, bandwidth), Raw signal (zero cross rate), and signal energy features were also employed.

In this paper a wide array of machine learning models is used, which includes models such as Support Vector Machine (SVM), Random Forest (RF), K-Nearest Neighbors (KNN), and XGBoost (Extreme Gradient Boosting) and Multi-layer Perceptron(MLP). This paper also employs some deep learning techniques such as VGG16 convolutional neural networks and Long Short-Term Memory (LSTM). The dataset used in this study is called the Fake-or-Real (FoR) dataset, which has for sub-datasets called FoR-rerec dataset, FoR-2sec dataset, For-norm dataset and For-original dataset where its description can be seen in table 2.6. The experiments were also conducted using noisy audio signals by adding synthetic noise to each audio sample in the for-2sec, for-norm, and for-rerec datasets. This approach retained both the original and noisy versions of the audio. For example, the for-2sec dataset originally contained 17,870 audio samples, and after adding noise, it expanded to 35,740 samples.

Table 2.6 Fake-Or-Real Dataset Description

Datasets	Size	Description
FoR-Rerec dataset	1.5 GB	It is a re-recorded version of the for-2-second dataset to simulate a scenario where an attacker sends an utterance through a voice channel
FoR-2sec dataset	1 GB	Contains audios based on FoR-Norm dataset, but with the files truncated at 2 seconds
FoR-Norm dataset	5.8 GB	The same files as FoR-Original dataset, bit balanced according to gender and the same sampling rate, volume, and channels for each
FoR-Original dataset	7.7 GB	The audios are collected from various sources, without any modification

For the FoR-rerec dataset the SVM model has shown to exhibit the highest accuracy of 98.83%, however for FoR-rerec noisy version, results depicts that MLP and SVM obtained the highest accuracy of 98.66% and 98.43%. In the FoR-2sec dataset however it is much easier for machine learning algorithms to process data in this form as it consists of audio with two seconds intervals. In this dataset the highest accuracy of both original and noisy version, SVM model outperforms the other ML models with it achieving the accuracy of 97.57% and

99.59%. However, for the FoR-norm dataset The Gradient Boosting classifier obtained the highest result of 92.60%, while the noisy version shows that XGB models perform the best with the accuracy of 94.25%. The results of these can be seen in the table 2.7 and table 2.8.

Table 2.7 Accuracy comparison for noisy audio signals using machine learning models.

Models	for-2sec	for-norm	for-rerec
SVM	97.57	71.54	98.83
MLP Classifier	94.69	86.82	98.79
Decision Tree	87.13	62.16	88.28
Extra Tree Classifier	94.61	91.46	96.87
Gaussian Naive Bayes	88.20	81.81	81.91
Ada Boost	90.23	88.40	87.67
Gradient Boosting	94.30	92.63	93.51
XGBoost	94.52	92.60	93.40
Linear Discriminant Analysis	89.50	91.35	87.56
Quadratic Discriminant Analysis	96.13	61.36	96.91

(Source: Hamza et al., 2022)

Table 2.8 Accuracy comparison for noisy audio signals using machine learning models.

Models	for-2sec	for-norm	for-rerec
SVM	99.59	75.21	98.43
MLP Classifier	99.49	89.22	98.66
Decision Tree	87.52	65.10	82.12
Extra Tree Classifier	97.91	90.19	96.25
Logistic Regression	87.53	86.28	88.00
Gaussian Naive Bayes	79.77	80.16	82.14
Ada Boost	85.28	91.35	83.89
Gradient Boosting	92.29	93.50	88.86
XGBoost	92.22	94.25	88.92
Linear Discriminant Analysis	87.05	90.52	85.88
Quadratic Discriminant Analysis	96.22	65.15	95.59

(Source: Hamza et al., 2022)

The for-original dataset, compiled from various sources, contains audio samples of different lengths, bit-rates, and noise levels. Traditional machine learning

models struggled with this dataset's complexity. To address this the authors would apply a transfer learning-based deep learning approach. MFCC visual features were extracted to train VGG-16 and LSTM models for deepfake audio classification. The VGG-16 model, using ImageNet weights and input shapes of $64 \times 64 \times 3$, achieved a testing accuracy of 93%, outperforming the LSTM model's 91%. The VGG-16 model also attained a validation accuracy of 94% and a validation loss of 0.14. Figures 2.11 illustrate the VGG-16 model's training and validation accuracy and loss, respectively.

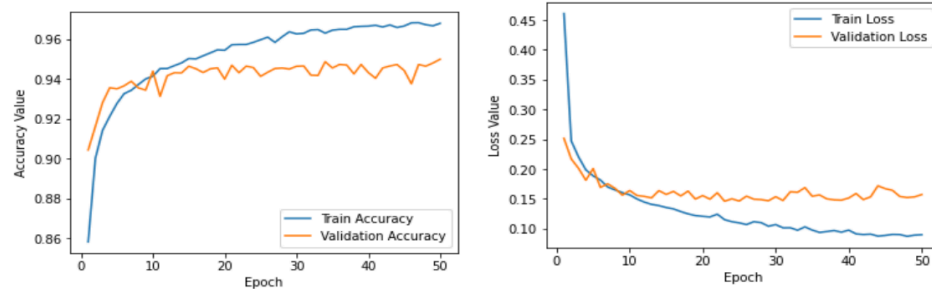


Figure 2.11 VGG-16 model's training and validation accuracy and loss

(Source: Hamza et al., 2022)

2.8.6 Observation on previous papers in detecting deepfake audio

Most papers use a combination of CNN alongside other techniques like LSTM (Altalahin et al., 2023) or standalone CNN (R. Anagha et al., 2023) and Qais et al., (2022). Nugroho & Winarno (2022) however used a mix of classical ML methods including Naive Bayes (NB), K-Nearest Neighbour (KNN), and Logistic Regression (LR), alongside a DNN approach. Hamza et al. (2022) however employed various ML models including Support Vector Machine (SVM), Random Forest (RF), K-Nearest Neighbors (KNN), XGBoost (Extreme Gradient Boosting), Multi-layer Perceptron (MLP), as well as deep learning techniques like VGG-16 and LSTM.

In terms of evaluation metrics, accuracy was most definitely the most common measure across studies. Some papers like that of R. Anagha et al. (2023) and

Qais et al. (2022) also included additional metrics such as Area Under the ROC curve (AUC). Nugroho & Winarno (2022) provided a comprehensive set of metrics including accuracy, precision, recall, and F1 score, offering a thorough evaluation of their model's performance.

The results indicated that Nugroho & Winarno (2022) achieved the highest performance with their DNN model, which significantly outperformed other methods across all metrics. Their DNN model achieved an accuracy of 96.5%, precision of 97.3%, recall of 96.5%, and an F1 score of 96.7%.

Regarding audio features and feature extraction, the Mel-Frequency Cepstral Coefficient (MFCC) was the most commonly used feature across studies due to its effectiveness in capturing the characteristics of speech signals. Some papers also employed additional features such as spectrograms, Short-Time Fourier Transform (STFT), Fast Fourier Transform (FFT), and other spectral features.

The datasets used in these studies varied but all focused on detecting spoofing or deepfake audio. The ASVspoof 2019 dataset was used by Altalahin et al. (2023) and R. Anagha et al. (2023), while Qais et al. (2022) used the ASVspoof 2017 dataset. Nugroho & Winarno (2022) utilized a custom dataset consisting of 10,000 samples labeled as REAL or FAKE. Hamza et al. (2022) used the Fake-or-Real (FoR) dataset, which included various sub-datasets with different characteristics.

Table 2.9 Observations on Previous Papers Used for Audio Deepfake Detection

Criteria	Altalahin et al., 2023	Qais et al., 2022	R. Anagha et al., 2023	Nugroho & Winarno, 2022	Hamza et al., 2022
Techniques Used	CNN-LSTM	CNN	CNN	NB, KNN, LR, DNN	SVM, RF, KNN, XGBoost, MLP, VGG-16, LSTM
Evaluation Metrics	Accuracy Precision Recall F1-score	Accuracy	Accuracy, AUC, Precision	Accuracy, Precision, Recall, F1 Score	Accuracy
Results	Accuracy : 88% Precision : 89% Recall : 87% F1-score : 87%	MFCC: 75.97%, Spectrogram: 68.9% STFT : 73.34% FFT: 71.48% Concated : 80.47%	Accuracy: 85%, AUC: 0.87, Average Precision: 0.90	DNN: Accuracy: 96.5%, Precision: 97.3%, Recall: 96.5%, F1: 96.7%	SVM: 91%, VGG-16: 93%
Audio Features used	MFCC	MFCC, Spectrogram, STFT, FFT	Mel-spectrogram	MFCC	MFCC, STFT, Spectral features, Raw signal, Signal energy
Dataset Used	ASVspoof 2019	ASVspoof 2017	ASVspoof 2019	Custom dataset (10,000 samples)	Fake-or-Real (FoR)

2.8.6 Discussion

In comparing various papers on deepfake audio detection, it is essential to evaluate not just on its performance but also the broader applicability such as audio features, dataset and techniques used. Although Nugroho & Winarno (2022) yield the best result compared to the rest, there are several considerations and potential limitations in their approach that might warrant a closer look, specifically their use of dataset. Nugroho & Winarno uses a custom dataset with 10,000 samples labeled as REAL or FAKE. While their results are impressive, the performance of their models may be highly tailored to this specific dataset. There is a possibility that their dataset does not fully capture the diversity and complexity of real-world deepfake audio, limiting the generalizability of their findings. In contrast, datasets like Fake-or-Real, ASVspoof 2017 and 2019, used by other studies, are well-established benchmarks that are designed to evaluate anti-spoofing systems comprehensively.

On the other hand, Hamza et al. (2022) and Qais et al. (2022) demonstrated the potential benefits of using a wide array of audio features. Audio features such as STFT, spectrogram, FFT and other spectral features were used compared to other studies which only used one audio feature albeit MFCC or mel-spectrogram. By using diverse features, they ensured that their models had a broader base of information to work with, potentially improving the robustness of their detection systems across different types of spoofing attacks.

Qais et al. (2022) presents a strong case for deepfake audio detection due to its use of a CNN model, which is well-suited for processing audio features. The study's exploration of multiple audio features and their combinations (with the concatenated data achieving 80.47% accuracy) demonstrates the effectiveness of using diverse features for robust classification. Despite using an older dataset (ASVspoof 2017), the findings and methodologies are relevant and can be adapted to newer datasets like ASVspoof 2021. This approach, combined with

the use of various audio features and the RNN approach that is suggested by Qais et al. (2022) makes it a viable reference for research focused on detecting deep fake audio.

In conclusion, while Hamza et al. (2022) provides a robust and diverse approach to deepfake detection using the FoR dataset, the direct relevance of Qais et al. (2022) to the ASVspoof series makes it the better choice for research utilising the ASVspoof 2021 dataset. The alignment of Qais et al. 's work with the ASVspoof datasets. With the combination of other audio feature such as mel-spectrogram as used in R. Anagha et al. (2023) research and evaluation metrics used in Altalihin et al. (2023) employed by Hamza et al. it may create an even more effective deeepfake detection systems

2.9 Summary

To conclude, this chapter mainly covers various information and components involved in developing a deep learning model for detecting deepfake images. The discussions are organized into several sections, addressing topics such as the definition of deepfake images, uses of deepfake images, impacts of deepfake images, analysis of differences between deepfake and real images and related works.

CHAPTER 3

RESEARCH METHODOLOGY

3.1 Research Framework

The research framework is a structured plan that outlines the key components and methodology towards this research project. It serves as a guide for the research process that ensures all aspects of the study are connected and are addressed systematically.

3.2 Preliminary Study

Table 3.1 Preliminary Study Phase

Phases	Activities	Deliverables
Preliminary study	Analyse the existing situation from previous papers to determine the problem for each paper Identify research question, objective, scope and significance of the project	1. Project Title 2. Research Background 3. Problem Statement 4. Research Question 5. Research Objectives 6. Research Scope 7. Research Significance

In this phase a review and analysis of existing literature will be done to further understand the current state of deepfake detection in audio. The Project Title “Deepfake detection in audio using machine learning” will help summarize the focus of the research. The Research Background however will provide much needed context by reviewing existing literature on deepfake detection methods in audio, with most of them differs in its use of audio features, methods, evaluation metrics and etc. The Problem Statement will clearly define the challenges that are often faced in this domain. The Research Question will guide the inquiry by posing a focused query. Research Objectives will outline the specific goals to be achieved through the study. Research Scope will set the boundaries and extent of the investigation. Finally, Research Significance will highlight the importance and potential impact of audio deepfake detection.

3.3 Knowledge Acquisition

Table 3.2 Knowledge Acquisition Phase

Phases	Activities	Deliverables
Knowledge acquisition	Conduct literature review through a research portal platform. Research Journal from databases: IEEE Xplore, Scopus, ScienceDirect, ACMDigitalLibrary. Study the methods and techniques has been developed	Literature Review Key audio features table

This phase involves performing an extensive literature review using research portals and databases such as IEEE Xplore, Scopus, ScienceDirect, and ACM Digital Library which is provided by the UiTM digital library. The primary deliverables include a comprehensive Literature Review Document and a list of key audio features relevant to deepfake detection. These deliverables provide a

detailed summary and critical analysis of existing methods and techniques and identify the most effective audio features for differentiating deepfaked audio from genuine recordings. This knowledge base will inform the subsequent phases of the project.

3.4 Data Acquisition

Table 3.3 Data Acquisition Phase

Phases	Activities	Deliverables
Data acquisition	Collect data from ASVspooF website (https://zenodo.org/records/4835108)	a dataset of a fake audio detection task comprising bona fide and spoofed utterances generated using TTS and VC algorithms.

This phase focuses on obtaining the ASVspooF 2021 dataset, which includes both bona fide (genuine) and spoofed (fake) utterances using Text-To-Speech (TTS) and Voice Conversion (VC) algorithms. The dataset consists of four files, and in each of those files contains the metadata of the audio that upholds information such as its ID, label, vocoder type etc. The dataset altogether accumulates to 34.5 gigabytes worth of audio files as can be seen in figure.

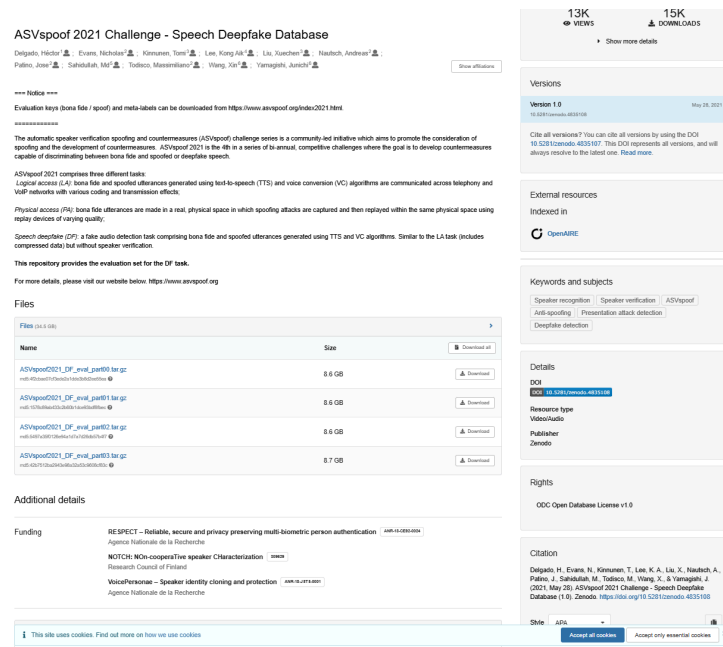


Figure 3.5 Data Pre-Processing

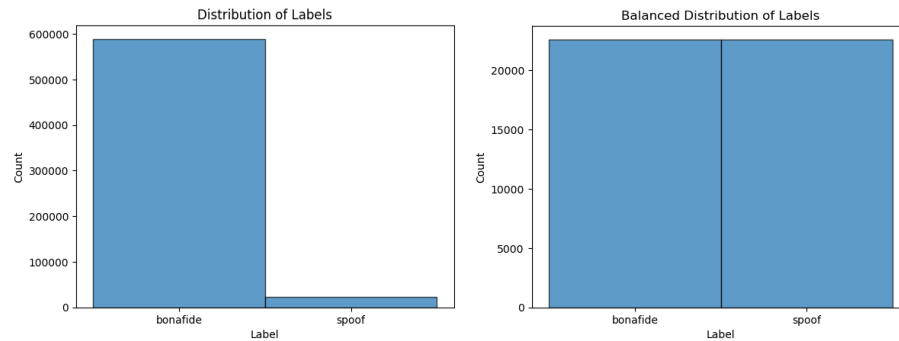
Table 3.4 Data Pre-Processing Phase

Phases	Activities	Deliverables
Data pre-processing	Data Under sampling Data Labelling Extract audio features Normalization Convert to image (CNN) Convert to time series (RNN) Stratified sampling	Pre-processed data

During the data pre-processing phase, the data that is obtained from the dataset will undergo several transformations to ensure its quality and is suitable for analysis or modelling. Due to the nature of CNN and RNN and the requirements that both of the models need to process audio data, CNN treats audio as image-like data, while RNN uses sequential nature of the audio for time-series

analysis. Data pre-processing for the most part will share the same steps, it will however later diverge to cater for the models' needs to achieve its input shape of each of the models.

First and foremost, both of the approaches for CNN and RNN start with compiling all the acquired files into 1 file making it convenient for further pre-processing. Once all of the audio files are collected it will go through a process called under sampling to combat data imbalance. Data imbalance occurs when a significant skew towards a particular category; as a result, algorithms trained on highly imbalanced data can end up over-classifying to the majority group (Daneshjou et al., 2021). During this process the data will be under sampled based on the lowest class, with it being spoof. This will result in the dataset originally being 611,829 to merely 45,234. This is to concentrate on the dataset quality, according to (Narwane & Sawarkar, 2022), they have stated that if the datasets are balanced, the system will predict accurate results and perform better, and vice versa.



After the data is undersampled as shown in figure , subsequent labelling will be done towards the data, where each of the 45,234 audio files will be labelled as spoof or bonafide based on the metadata that is given.

```
File Name: DF_E_3973141.flac, File Identifier: DF_E_3973141
Matched: D:\audio_files\flac2\DF_E_3973141.flac with label bonafide
File Name: DF_E_3890571.flac, File Identifier: DF_E_3890571
Matched: D:\audio_files\flac2\DF_E_3890571.flac with label bonafide
File Name: DF_E_2677721.flac, File Identifier: DF_E_2677721
Matched: D:\audio_files\flac\DF_E_2677721.flac with label spoof
File Name: DF_E_3816888.flac, File Identifier: DF_E_3816888
Matched: D:\audio_files\flac\DF_E_3816888.flac with label spoof
```

Once the labelling is done the audio files will be loaded and the extraction of Mel-frequency cepstral coefficients (MFCCs) will be done with the help of the librosa library, which is a python package for music and audio analysis.

```
# Audio to MFCC conversion function
def audio_to_mfcc(audio_file, sr=22050, n_mfcc=13):
    """Convert audio to MFCC."""
    audio, sample_rate = librosa.load(audio_file, sr=sr)
    mfcc = librosa.feature.mfcc(y=audio, sr=sample_rate, n_mfcc=n_mfcc)
    return mfcc
```

The pre-processing after this will cater towards the need for each model starting from CNN where with the help of OpenCV it will undergo resizing where the extracted MFCCs will be resized into a fixed 2D shape being (128,128).

```
# Resize the MFCC to a fixed size function
def resize_spectrogram(spectrogram, target_size=(128, 128)):
    """Resize the MFCC to a fixed size."""
    return cv2.resize(spectrogram, target_size)
```

Normalization is then done towards the MFCC; this is to ensure that the values of MFCC lie within the range of [0,1], ensuring that the values of the MFCC stays consistent; this is done by dividing the maximum value of the resized MFCC. The labels are then converted into binary format.

```
# Preprocessing the audio files
def preprocess_audio_files(audio_files, labels, target_size=(128, 128)):
    """Convert audio files to MFCCs, resize, and normalize them."""
    mfcc_data = []
    label_data = []

    for audio_file, label in zip(audio_files, labels):
        mfcc = audio_to_mfcc(audio_file)
        resized_mfcc = resize_spectrogram(mfcc, target_size)

        # Safe normalization: Check if the max value is 0
        max_value = np.max(resized_mfcc)
        if max_value > 0:
            normalized_mfcc = resized_mfcc / max_value
        else:
            normalized_mfcc = resized_mfcc # Leave it as is if the max is 0

        mfcc_data.append(normalized_mfcc)
        label_data.append(1 if label == 'spoof' else 0) # Binary label (1: spoof, 0: bonafide)

    return np.array(mfcc_data), np.array(label_data)
```

Lastly, for CNN a channel dimension will then be added and the MFCCs will be converted into a 3D array with dimensions (num_samples, 128, 128, 1) for CNN input.

```
# Preprocess the valid audio files and labels
X, y = preprocess_audio_files(valid_audio_files, valid_labels, target_size=(128, 128))

# Add a channel dimension for CNN
X = np.expand_dims(X, axis=-3)
```

However, for RNN, padding or truncating will be done where the extracted MFCCs are padded to a fixed number of time steps, this is done to preserve the sequence data structure.

```
def pad_or_truncate(mfcc, max_time_steps=400):
    if mfcc.shape[1] > max_time_steps:
        return mfcc[:, :max_time_steps]
    elif mfcc.shape[1] < max_time_steps:
        padding = max_time_steps - mfcc.shape[1]
        return np.pad(mfcc, ((0, 0), (0, padding)), mode='constant')
    return mfcc
```

Normalization is also done towards RNN, but rather than dividing with the maximum value like CNN, RNN utilizes StandardScaler to normalize each feature dimension to have zero mean and unit variance, normalization is done differently for RNN due to its different input shape compared to CNN. The preprocessed audio file is then converted into its intended shape so it is suitable to feed into the RNN model.

```
# Preprocessing function with normalization
def preprocess_audio_files_rnn(audio_files, labels, n_mfcc=13, max_time_steps=400):
    mfcc_data = []
    label_data = []
    scaler = StandardScaler()

    for audio_file, label in zip(audio_files, labels):
        mfcc = audio_to_mfcc(audio_file, n_mfcc=n_mfcc)
        mfcc = pad_or_truncate(mfcc, max_time_steps=max_time_steps).T
        mfcc = scaler.fit_transform(mfcc)
        mfcc_data.append(mfcc)
        label_data.append(1 if label == 'spoof' else 0)

    return np.array(mfcc_data), np.array(label_data)

X, y = preprocess_audio_files_rnn(valid_audio_files, valid_labels)
print(f"Shape of X: {X.shape}")
print(f"Shape of y: {y.shape}")
```

The table below shows the difference in the pre-processing steps done by both CNN and RNN.

Pre-processing steps	CNN	RNN
MFCC Processing	Resized to (128, 128) using cv2.resize	Padded/Truncated to max_time_steps=400
Normalization	Max value scaling (0 to 1)	StandardScaler (zero mean, unit variance)
Data Representation	2D MFCCs as images	MFCCs as Sequential data
Input Shape	(num_samples, 128, 128, 3)	(num_samples, max_time_steps, n_mfcc)

Before the data is fed into the models, the data set will be splitted into a training and a test set. The method used for the splitting is stratified sampling, this method is chosen because stratified sampling can be applied to guarantee that the number of samples for each class is balanced (Jeyabharathy Sadaiyandi et al., 2023). This is done so that the dataset stays consistent during training when it is trained with different models.

```
# Stratified 70:30 train-test split
def stratified_split_70_30(X, y, test_size=0.3, random_state=42):
    """Split dataset into stratified 70/30 sets."""
    strat_split = StratifiedShuffleSplit(n_splits=1, test_size=test_size, random_state=random_state)

    for train_idx, test_idx in strat_split.split(X, y):
        X_train, y_train = X[train_idx], y[train_idx]
        X_test, y_test = X[test_idx], y[test_idx]

    # Ensure that the training set has balanced classes
    class_counts = np.bincount(y_train)
    min_class_count = min(class_counts)
    spoof_indices = np.where(y_train == 1)[0][:min_class_count]
    bonafide_indices = np.where(y_train == 0)[0][:min_class_count]
    balanced_train_indices = np.concatenate([spoof_indices, bonafide_indices])
    X_train_balanced, y_train_balanced = X_train[balanced_train_indices], y_train[balanced_train_indices]

    return X_train_balanced, X_test, y_train_balanced, y_test

# Perform the stratified 70/30 split
X_train, X_test, y_train, y_test = stratified_split_70_30(X, y, test_size=0.3, random_state=42)

# Print the distribution of classes to verify stratification
print(f"Training set distribution: {np.bincount(y_train)}")
print(f"Test set distribution: {np.bincount(y_test)}")
```

3.6 Model Development

Table 3.6 Model Development Phase

Phases	Activities	Deliverables
Model development	Develop a deepfake audio detection model using CNN and RNN	Train CNN and RNN model for audio deepfake detection
	Train and test model	
	Hyperparameter tuning	

The CNN structure is done by where the input shape will go through 3 convolutional layers with filters of 16, 32 and 64 for each filter respectively, where it will then be flattened and passed into a dense layer with a layer of 128 units with an activation function and a dropout layer. Finally an output layer with sigmoid activation function for classification.

```
# Define the CNN architecture
def build_cnn(input_shape):
    model = models.Sequential()

    # 1st Convolutional Layer
    model.add(layers.Conv2D(16, kernel_size=(3, 3), activation='relu', input_shape=input_shape))
    model.add(layers.MaxPooling2D(pool_size=(2, 2)))

    # 2nd Convolutional Layer
    model.add(layers.Conv2D(32, kernel_size=(3, 3), activation='relu'))
    model.add(layers.MaxPooling2D(pool_size=(2, 2)))

    # 3rd Convolutional Layer
    model.add(layers.Conv2D(64, kernel_size=(3, 3), activation='relu'))
    model.add(layers.MaxPooling2D(pool_size=(2, 2)))

    # Flatten the output and add fully connected layers
    model.add(layers.Flatten())
    model.add(layers.Dense(128, activation='relu'))
    model.add(layers.Dropout(0.4))

    # Output layer (binary classification)
    model.add(layers.Dense(1, activation='sigmoid'))

    # Compile the model
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

    return model
```

For RNN structure, either by using LSTM and GRU layers, they will be stacked in three layers with units of 128, 64, and 32 respectively, followed by batch normalization. Furthermore, similar to the CNN architecture above it will also be flattened and be passed through a dense output layer for binary classification with a sigmoid activation function.

```
def build_rnn_model(input_shape):
    model = Sequential([
        Input(shape=input_shape),
        LSTM(128, return_sequences=True, activation='tanh'),
        Dropout(0.3),
        BatchNormalization(),
        LSTM(64, return_sequences=True, activation='tanh'),
        Dropout(0.3),
        BatchNormalization(),
        LSTM(32, activation='tanh'),
        Dropout(0.3),
        Dense(32, activation='relu'),
        Dropout(0.3),
        Dense(1, activation='sigmoid')
    ])
    return model
```

Both models will be trained with 100 epochs with early stopping being applied for both models where it will monitor the validation loss. If the validation loss does not improve for 5 consecutive epochs the training will stop prematurely, this is done to prevent overfitting and can save computation time. Furthermore, 20% of the training data is reserved for validation during training.

```
# Early stopping
early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

# Train the model
history = rnn2_model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=64,
    callbacks=[early_stopping],
    verbose=1
)
```

3.7 Model Evaluation

Table 3.7 Model Evaluation Phase

Phases	Activities	Deliverables
Model evaluation	Evaluate the performance of deepfake audio detection	Performance evaluation

In the Model Evaluation phase, the primary focus is on assessing the performance of the CNN and RNN models that are designed for audio deepfake detection. Understanding how models will be evaluated is important for understanding how well the model may generalize in the real world, in this case how well it will detect deepfaked audio in the real world. The evaluation is done with the help of the confusion matrix where it will help calculate measures such as accuracy, precision, and recall and f1-score which can give a better idea of the model's performance,

Accuracy

Accuracy is a measure for how many correct predictions are done towards the model. Accuracy is a good base metric to measure the performance of a model

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

Precision

Precision on the other hand tells how many of the correctly predicted cases actually turned out to be positive. Precision is normally done to see whether the

model is reliable or not. Precision is particularly useful in cases where False Positive is a higher concern than False Negative.

$$\text{Precision} = \frac{TP}{TP+FP}$$

Recall

Recall tells on how many of the actual positive cases were able to predict correctly with the model. Recall is a useful metric in cases where False Negatives outweighs false positives.

$$\text{Recall} = \frac{TP}{TP+FN}$$

F1-Score

F1-score combines precision and recall into a single metric to balance their trade-off. It provides a better sense of a model's overall performance.

$$\text{F1-Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

3.8 Prototype Development

Table 3.9 Prototype Development Phase

Phases	Activities	Deliverables
Prototype Development	Develop prototype	Audio deepfake detection

In the prototype phase, said Audio Deepfake Detection prototype will be built. As the prototype will be web-based this involves writing necessary code and integrating all the components in detecting audio deepfake in the prototype. The goal of this phase is to develop a prototype that can further be tested and be evaluated in the following phase

3.9 Testing

Table 3.10 Testing Phase

Phases	Activities	Deliverables
Testing	Test functionality and usability of prototype	Result of the testing

The next phase after prototype development will be testing which will be done by performing various tests to ensure the prototype functionality and usability and making necessary adjustments. The goal of this phase is to refine the prototype until it meets the desired standards and requirements.

3.10 Documentation

Table 3.11 Documentation phase

Phases	Activities	Deliverables
Documentation	Writing a report	Report

In the documentation phase it involves thoroughly documenting the entire research process, capturing the methodology, development, and evaluation phases. It encompasses the design, implementation, and testing stages, providing

a comprehensive overview of the work undertaken. The report summarises key findings, challenges encountered, and solutions implemented, ensuring a clear and structured account of the research journey and its outcomes.

3.11 Summary

This chapter explains how the research process will be conducted in order to achieve the research objectives. This will be achieved by following the research framework which employs 10 different phases that being preliminary study, knowledge acquisition, data acquisition, data pre-processing, model development, model evaluation, prototype development, testing and documentation.

CHAPTER 4

RESULTS AND FINDINGS

4.1 Introduction

The chapter will present the results and findings of the CNN and RNN models where the performance of each experiment is focused upon to further get better results based on the evaluation methods.

4.2 CNN Model Experiments

Experiments are conducted by following a base system architecture and later the relevant parameters are changed to find the most optimized hyper parameter. Parameters that are subject for further experiments are learning rate, optimizer, activation function and dropout rate. Parameters such as batch size, dense layer, kernel size, etc. were not specifically chosen as during early experiments it has shown that it does not noticeably impact the results of the models. The table below describes the base parameters that is used for the CNN model

Parameters	Values
Split Ratio	80:20
Epochs	100 (with early stopping applied, patience = 10)
Batch Size	128
Learning Rate	0.001
Input Shape	128,128,1
Dropout Rate	0.4

Optimizer	Adam
Activation Function	ReLu

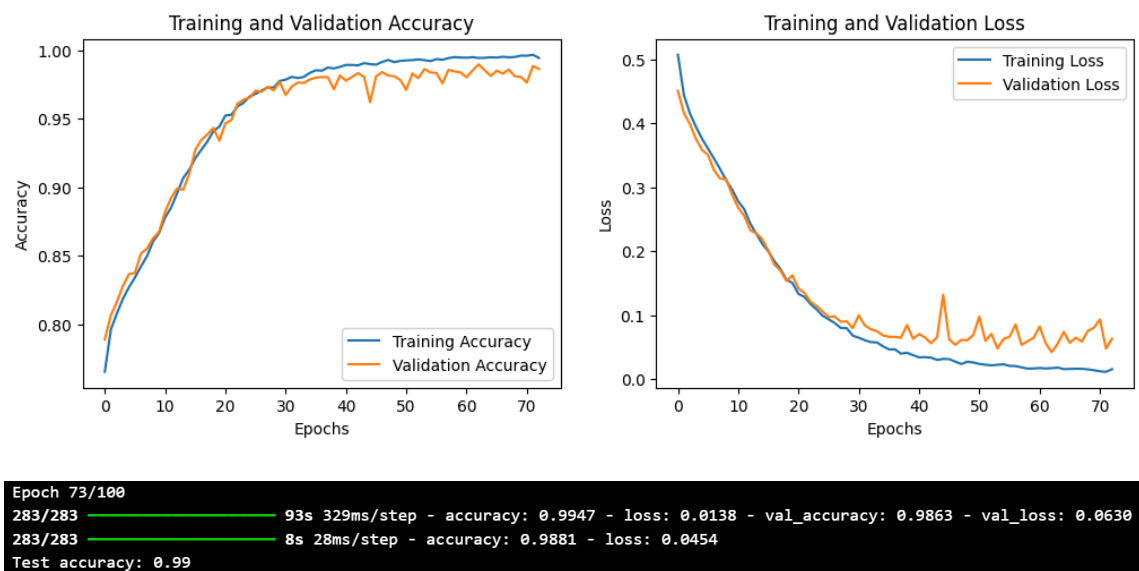
4.2.1 Learning Rate

Different values of learning rates are experimented upon the models to see how it responds upon adjustments. Learning rate values that were experimented on were 0.01, 0.001 and 0.0001. These values were tested on both 80/20 and 70/30 data split.

Summary of learning rate Experiment

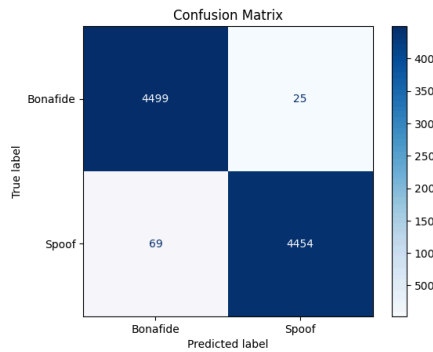
	EX1	EX2	EX3	EX4	EX5
Split	80/20	80/20	80/20	70/30	70/30
Input Shape	128,128,3	128,128,3	128,128,3	128,128,3	128,128,3
Activation function	ReLu	ReLu	ReLu	ReLu	ReLu
Dense Layer	128	128	128	128	128
Dropout Layer	0.4	0.4	0.4	0.4	0.4
Kernel Size (Conv Layers)	3,3	3,3	3,3	3,3	3,3
Learning Rate	0.01	0.001	0.0001	0.01	0.001
Optimizer	Adam	Adam	Adam	Adam	Adam
Pooling Size	2,2	2,2	2,2	2,2	2,2
Patience	10	10	10	10	10
Batch Size	128	128	128	128	128
Epoch	30/100	21/100	73/100	48/100	22/100
Test Accuracy	0.95	0.9813	0.9881	0.9228	0.9797
Loss	0.1515	0.0595	0.0454	0.185	0.0641
Precision	0.96	0.99	0.99	0.94	0.99
Recall	0.93	0.98	0.98	0.9	0.97
F1 Score	0.95	0.98	0.99	0.92	0.98

Based on the experiments done by tweaking the learning rate it has been found that the learning rate that achieved the highest accuracy is 0.001 on a 80:20 split. A learning rate of 0.01 on a 70:30 however achieved the lowest accuracy compared to the others. Figure shows the results of the best performing model during the learning rate experiments



From the figure above it shows that this model which lasts for 73 epochs has achieved a training accuracy of 99.47%, which is relatively close to the validation accuracy of 98.63%, with the validation loss for training and

validation both being 0.0138 and 0.0454 respectively. The test accuracy has also achieved an accuracy of 98.81% and a loss of 0.0454. Furthermore, from the figure it can be seen that the accuracy for both training and validation increases slowly before ultimately reaching its plateau around the 30th epoch. For the training and validation loss, it can be seen that they follow the same trajectory, however validation loss later steers away from training loss around also around the 30th epoch. This along with the performance based on the metrics in figure suggests that the model shows no sign of overfitting and is able to distinguish spoof audio from bonafide audio with a very high accuracy.



Precision: 0.99
Recall: 0.98
F1 Score: 0.99

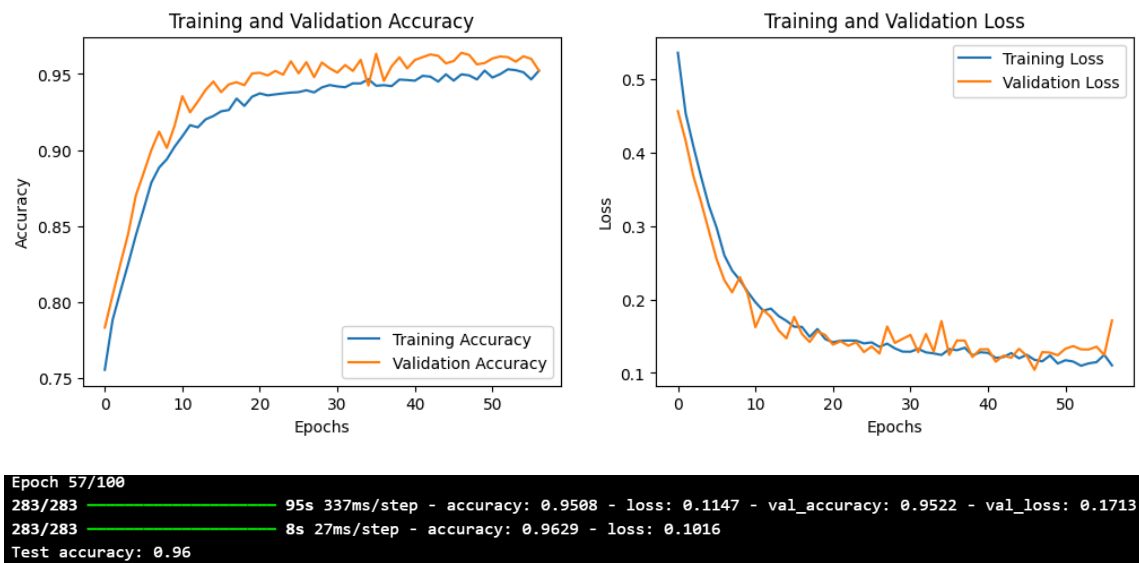
4.2.2 Optimizer

In this experiment rather than using Adam as the optimizer, AdamW is substituted to be the optimizer. With the introduction of AdamW a new parameter called weight decay, this optimizer along with its weight decay are then experimented on with the CNN model.

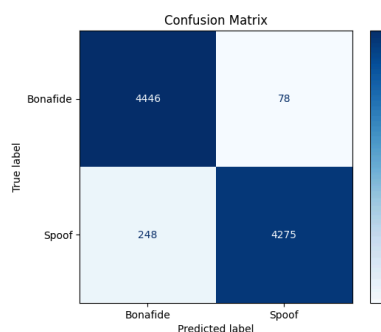
Summary of optimizer Experiment

	EX1	EX2	EX3	EX4	EX5	EX6
Split	80/20	80/20	80/20	80/20	70/30	70/30
Input Shape	128,128, 3	128,128, 3	128,128, 3	128,128, 3	128,128, 3	128,128, 3
Activation function	ReLu	ReLu	ReLu	ReLu	ReLu	ReLu
Dense Layer	128	128	128	128	128	128
Dropout Layer	0.4	0.4	0.4	0.4	0.4	0.4
Learning Rate	0.01	0.01	0.01	0.01	0.01	0.01
Weight Decay	0.01	0.001	0.1	0.0001	0.01	0.001
Optimizer	AdamW	AdamW	AdamW	AdamW	AdamW	AdamW
Pooling Size	2,2	2,2	2,2	2,2	2,2	2,2
Patience	10	10	10	10	10	10
Batch Size	128	128	128	128	128	128
Epoch	57/100	45/100	48/100	45/100	49/100	37/100
Test Accuracy	0.9629	0.9253	0.9334	0.9439	0.9619	0.9265
Loss	0.1016	0.2225	0.1766	0.161	0.1082	0.1527
Precision	0.98	0.94	0.94	0.94	0.98	0.91
Recall	0.95	0.91	0.93	0.94	0.95	0.96
F1 Score	0.96	0.93	0.94	0.95	0.96	0.94

Experiments done with AdamW optimizer and its weight decay shows that a weight decay of 0.01 on a 80:20 split performs the best, while a weight decay of 0.001 on a 80:20 split performs the worst. Figure shows the results of the best performing model during the optimizer experiments.



The Figure above shows that AdamW with a weight decay of 0.01 on a 80:20 split achieved an accuracy of 95.08% and 95.22% and a loss of 0.1147 and 0.1713 for training and validation respectively. The testing accuracy achieved from this is 96.29% and test loss of 0.1016 with it lasting for a total of 57 epochs. From the figure it can be seen that the graph quickly reaches its plateau around epoch 15 and above. The number of epochs suggests that the model takes time to achieve its optimal performance.



Precision: 0.98
Recall: 0.95
F1 Score: 0.96

4.2.3 Activation Function

This next experiment focuses on the activation function of LeakyRelu rather than Relu. With the introduction of LeakyRelu, the alpha parameter is introduced, the activation function is then tested with both optimizer Adam and AdamW. A total of 6 experiments were conducted all together

Summary of activation function Experiment

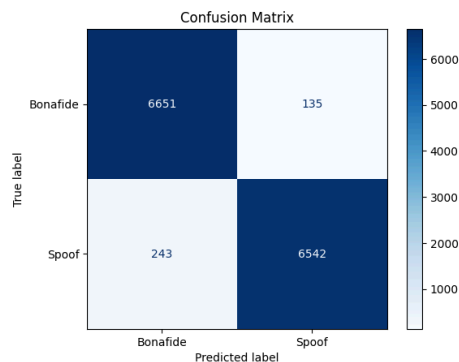
	EX1	EX2	EX3	EX4	EX5	EX6
Split	80/20	80/20	80/20	70/30	70/30	70/30
Input Shape	128,128,3	128,128,3	128,128,3	128,128,3	128,128,3	128,128,3
Activation function	LeakyRelu	LeakyRelu	LeakyRelu	LeakyRelu	LeakyRelu	LeakyRelu
Dense Layer	128	128	128	128	128	128
Dropout Layer	0.4	0.4	0.4	0.4	0.4	0.4
Learning Rate	0.01	0.01	0.01	0.01	0.01	0.01
Weight Decay	0.1	0.1	-	-	0.1	0.1
Alpha	0.1	0.01	0.01	0.01	0.1	0.01
Optimizer	AdamW	AdamW	Adam	Adam	AdamW	AdamW
Pooling Size	2,2	2,2	2,2	2,2	2,2	2,2
Patience	10	10	10	10	10	10
Batch Size	128	128	128	128	128	128
Epoch	23/100	15/100	21/100	39/100	29/100	24/100
Test Accuracy	0.9486	0.8678	0.9146	0.972	0.9629	0.9251
Loss	0.1367	0.3022	0.2021	0.0973	0.1219	0.1928
Precision	0.95	0.91	0.9	0.98	0.96	0.94
Recall	0.95	0.82	0.94	0.96	0.95	0.9
F1 Score	0.95	0.87	0.92	0.97	0.96	0.92

The experiments conducted during the activation function of LeakyRelu with alpha set to 0.01 and paired with Adam optimizer achieved the highest testing accuracy. However, for all of the experiments the training validation and accuracy will suddenly dip a substantial amount and come back up. This can also be seen from the best model during the experiment in the figure below.



```
Epoch 39/100
248/248 93s 374ms/step - accuracy: 0.9802 - loss: 0.0590 - val_accuracy: 0.9682 - val_loss: 0.1352
425/425 10s 23ms/step - accuracy: 0.9720 - loss: 0.0973
Test accuracy: 0.97
```

From figure 1.1 the model lasting for 39 epochs achieved an accuracy of 98.02%, 96.82% and 97.2% for training, accuracy and testing, with the loss 0.0590, 0.1352 and 0.0973 respectively. Despite the high accuracy, during training it can be seen that the model drastically dipped around the 19th epoch but 4 epoch later it immediately went back up to where it peaked before stabilizing.



```
Precision: 0.98
Recall: 0.96
F1 Score: 0.97
```

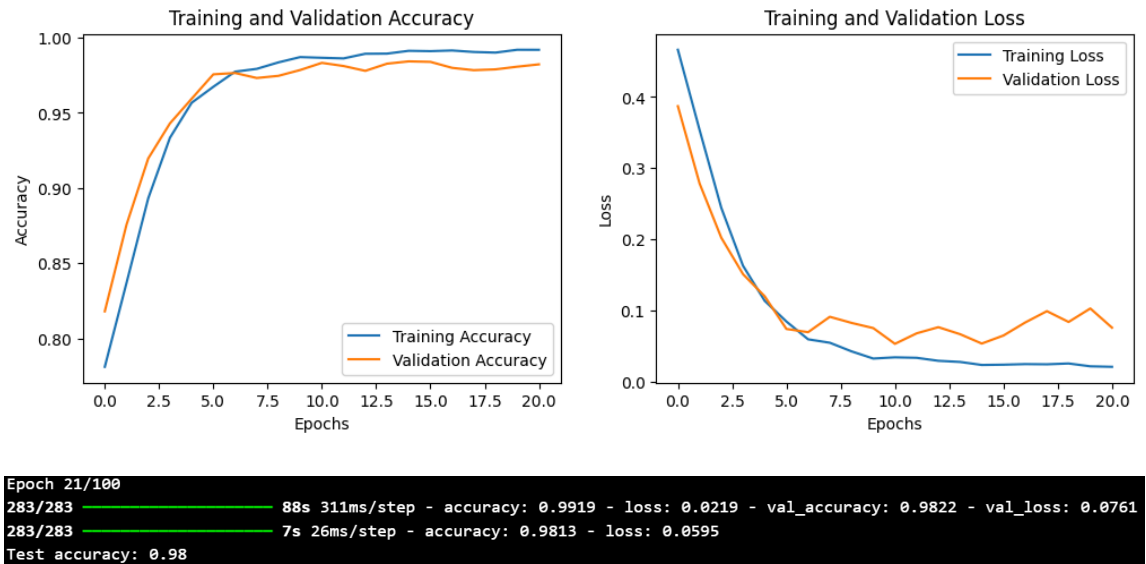
4.2.4 Dropout Rate

Dropout rate experiment consists of experiments on different rates of 0.9, 0.7, 0.4 and 0.1. This is to see how the CNN model reacts to the change of dropout rate.

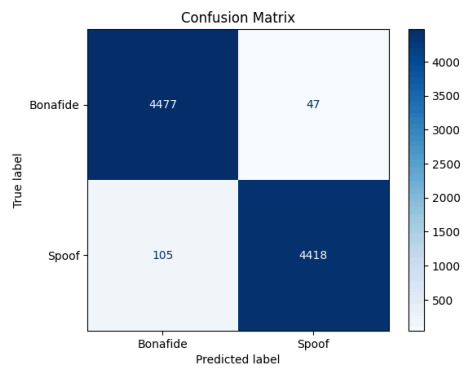
Summary of dropout rate Experiment

	EX1	EX2	EX3	EX4	EX5
Split	80/20	80/20	80/20	80/20	70/30
Input Shape	128,128,3	128,128,3	128,128,3	128,128,3	128,128,3
Activation function	ReLu	ReLu	ReLu	ReLu	ReLu
Dense Layer	128	128	128	128	128
Dropout Layer	0.9	0.1	0.7	0.4	0.9
Learning Rate	0.001	0.001	0.001	0.001	0.001
Optimizer	Adam	Adam	Adam	Adam	Adam
Pooling Size	2,2	2,2	2,2	2,2	2,2
Patience	10	10	10	10	10
Batch Size	128	128	128	128	128
Epoch	58/100	17/100	29/100	21/100	64/100
Test Accuracy	0.9616	0.9803	0.9834	0.9813	0.9709
Loss	0.1247	0.075	0.0584	0.0595	0.0883
Precision	0.97	0.99	0.99	0.99	0.99
Recall	0.96	0.97	0.97	0.98	0.96
F1 Score	0.96	0.98	0.98	0.98	0.97

The experiments with different dropout rates shows that a dropout rate of 0.4 on a 80:20 split performs the best while a dropout rate of 0.9 on a 70:30 split performs the worst. Coincidentally, due to the nature of how the experiments are conducted, the results from this experiment are the same as the base parameter. Figure shows the results of the best performing model during the learning rate experiments



From the figure above it shows that this model which lasts for 21 epochs has achieved a training accuracy of 99.19%, which is relatively close to the validation accuracy of 98.22%, with the validation loss for training and validation both being 0.0219 and 0.0761 respectively. The test accuracy has also achieved an accuracy of 98.13% and a loss of 0.0595. Furthermore, from the figure it can be seen that the accuracy for both training and validation increases substantially for the first few epochs before reaching its plateau around epoch number 6. This can also be said for the training and validation loss, however it can be seen that compared to training loss, validation loss steers away from training loss and suddenly fluctuates slightly after 8 epochs. This along with the performance based on the metrics in figure suggests that the model shows no sign of overfitting and with the early stoppage being stopped as early as the 21st epoch indicating that it reached its optimal performance rather quickly.



Precision: 0.99
Recall: 0.98
F1 Score: 0.98

4.2.5 Summary

Based on all of the experiments that are conducted for CNN by fine tuning all the hyperparameters that are involved it has shown that a learning rate of with performs the best, however it has shown that by using LeakyRelu as an activation function, whether with Adam or AdamW optimizer as its pairing, there would be an instance where the accuracy and loss will randomly spike and promptly goes back to normal.

4.3 RNN Model Experiment

Similar to how Experiments are conducted with CNN, RNN also follows a base system architecture and later the relevant parameters are changed to find the most optimized hyper parameter. Furthermore, experiments for RNN used two different layers , LSTM and GRU. The table below describes the base parameters that are used for the RNN for both LSTM and GRU models.

4.4 RNN LSTM Model Experiment

The first sets of experiments done with RNN is done using LSTM layers, where hyperparameters such as learning rate, dropout layer and optimizer were carefully optimized.

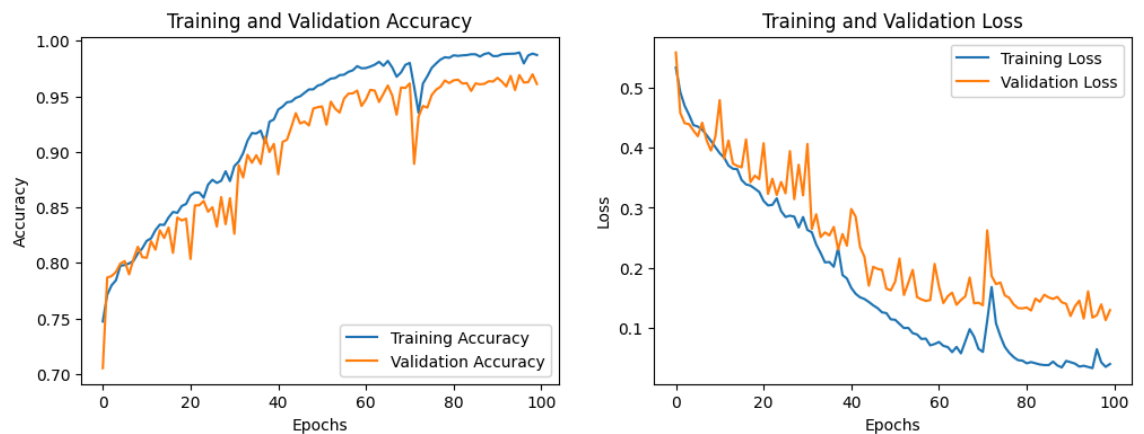
4.4.1 Learning Rate

This experiment focuses on experimenting learning rate with RNN LSTM architecture, with learning rate values of 0.01, 0.001 and 0.0001 are tested on a 80:20 split.

Summary of learning rate Experiment

	EX1	EX2	EX3
Split	80/20	80/20	80/20
Dropout Layer (LSTM)	0.3	0.3	0.3
Activation function (LSTM)	Tanh	Tanh	Tanh
Dense Layer	128	128	128
Activation Function (Dense Layer)	ReLu	ReLu	ReLu
Dropout (Dense layer)	0.3	0.3	0.3
Activation Function (Output Layer)	sigmoid	sigmoid	sigmoid
Learning Rate	0.0001	0.01	0.001
Optimizer	Adam	Adam	Adam
Patience	10	10	10
Batch Size	64	64	64
Epoch	53	27	100
Test Accuracy	0.8583	0.8160	0.9581
Loss	0.3130	0.3915	0.1303
Precision	0.90	0.83	0.95
Recall	0.80	0.79	0.96
F1 Score	0.85	0.81	0.96

The experiments conducted show that with a learning rate of 0.001 with optimizer Adam has achieved the highest accuracy, while a learning rate of 0.01 appears to be the lowest out of the three in terms of overall performance. The best model can be seen in figure



```
Epoch 100/100
453/453 [=====] - 72s 159ms/step - loss: 0.0408 - accuracy: 0.9871 - val_loss: 0.1302 - val_accuracy: 0.9610
Test Loss: 0.1303
Test Accuracy: 0.9581
```

The figure above shows that the model above has not satisfied the stopping condition as it has gone through the whole 100 epochs, which makes the convergence of the model slow. However, the model achieved a high accuracy of 98.71%, 96.10% and 95.81% for training, validation and testing and a loss of 0.0408, 0.1302 and 0.1303 respectively. Furthermore, a slight dip can be seen around 73 epochs where it swiftly recovered back to normal.

```
Confusion Matrix:
[[4319  205]
 [ 174 4349]]
```

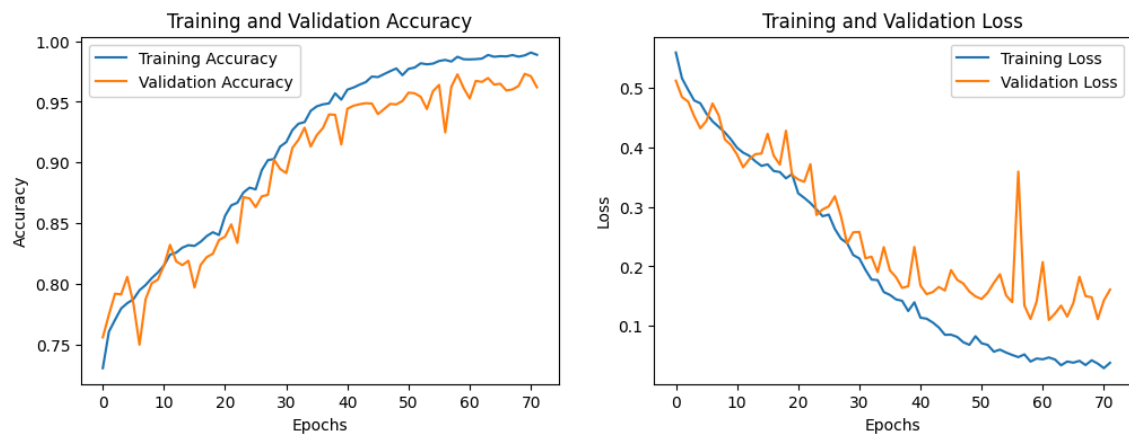
	precision	recall	f1-score
0	0.96	0.95	0.96
1	0.95	0.96	0.96

4.4.2 Dropout Layer

For the dropout layer experiment, both dropout rate from the LSTM layer and dense layer with the same value. The values being 0.3, 0.5 and 0.7 were tested on an 80:20 split.

	EX1	EX2	EX3
Split	80/20	80/20	80/20
Dropout Layer (LSTM)	0.3	0.5	0.7
Activation function (LSTM)	Tanh	Tanh	Tanh
Dense Layer	128	128	128
Activation Function (Dense Layer)	ReLu	ReLu	ReLu
Dropout (Dense layer)	0.3	0.5	0.7
Activation Function (Output Layer)	sigmoid	sigmoid	sigmoid
Learning Rate	0.001	0.001	0.001
Optimizer	Adam	Adam	Adam
Patience	10	10	10
Batch Size	64	64	64
Epoch	27	72	47
Test Accuracy	0.8160	0.9678	0.9038
Loss	0.3915	0.1114	0.2338
Precision	0.83	0.97	0.96
Recall	0.79	0.96	0.85
F1 Score	0.81	0.97	0.90

From the experiments conducted it is observed that with a dropout rate of 0.5 on both LSTM and Dense Layer achieved the highest accuracy, while a dropout rate of 0.3 performs significantly low compared to the other two models with a test accuracy of 83%.. The performance of the stated best model of dropout rate of 0.5 is shown in figure



```
Epoch 72/100
453/453 [=====] - 56s 124ms/step - loss: 0.0384 - accuracy: 0.9887 - val_loss: 0.1611 - val_accuracy: 0.9620
Test Loss: 0.1114
Test Accuracy: 0.9678
```

The graph above indicates that the model performs well and is run through 72 epochs before stopping. The model performs well as it achieved a training accuracy of 98.87%, validation accuracy of 96.20% and a testing accuracy of 96.78% with losses of 0.034, 0.1611 and 0.1114 respectively. In the graph it can be seen that the training and validation accuracy generally follows each other with the same pattern, however around 40 epochs and further it diverges significantly, more so in loss rather than accuracy.

Confusion Matrix:
[[4411 113]
[178 4345]]

	precision	recall	f1-score
0	0.96	0.98	0.97
1	0.97	0.96	0.97

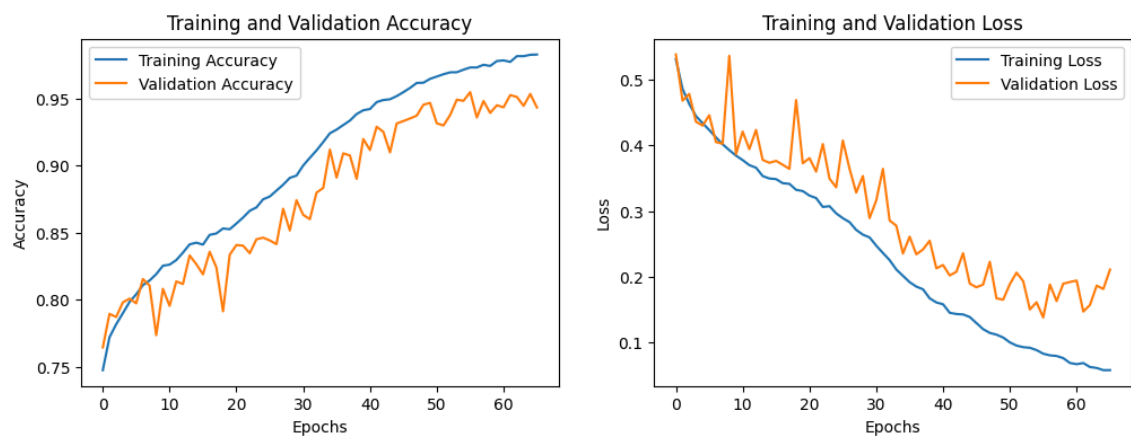
4.4.3 Optimizer

Rmsprop is used to be experimented on as the optimizer rather than Adam, where the three different learning rates are changed to 0.01, 0.001 and 0.0001 to see how they perform on a 80:20 split.

Summary of optimizer Experiment

	EX1	EX2	EX3
Split	80/20	80/20	80/20
Dropout Layer (GRU)	0.3	0.3	0.3
Activation function (GRU)	Tanh	Tanh	Tanh
Dense Layer	128	128	128
Activation Function (Dense Layer)	ReLu	ReLu	ReLu
Dropout (Dense layer)	0.3	0.3	0.3
Activation Function (Output Layer)	sigmoid	sigmoid	sigmoid
Learning Rate	0.0001	0.01	0.001
Optimizer	Rmsprop	Rmsprop	Rmsprop
Patience	10	10	10
Batch Size	64	64	64
Epoch	90	21	66
Test Accuracy	0.8819	0.8212	0.9542
Loss	0.2859	0.4013	0.1400
Precision	0.91	0.86	0.98
Recall	0.84	0.76	0.93
F1 Score	0.88	0.81	0.95

Based on the summary of the optimizer experiment above, it can be concluded that Rmsprop with a learning rate of 0.001 performs the greatest, while with a Rmsprop learning rate 0.01 performs the worst compared to the rest. The figure below shows the performance of said Rmsprop with the learning rate of 0.001.



```
Epoch 66/100
453/453 [=====] - 58s 129ms/step - loss: 0.0581 - accuracy: 0.9827 - val_loss: 0.2111 - val_accuracy: 0.9432
Test Loss: 0.1400
Test Accuracy: 0.9542
```

The above figure shows that the Rmsprop optimizer with the learning rate of 0.001 has shown a high accuracy and low loss all across the board. With its accuracy of training, validation and test reaches 98.27%, 94.32% and 95.42% respectively. The graph shows that between training and validation small gaps between them appears across all 66 epochs.

Confusion Matrix:
[[4420 104]
[310 4213]]

	precision	recall	f1-score
0	0.93	0.98	0.96
1	0.98	0.93	0.95

4.5 RNN GRU Model Experiment

The second sets of experiments done with RNN is done using GRU layers instead, where hyperparameters such as learning rate, dropout layer and optimizer were carefully optimized

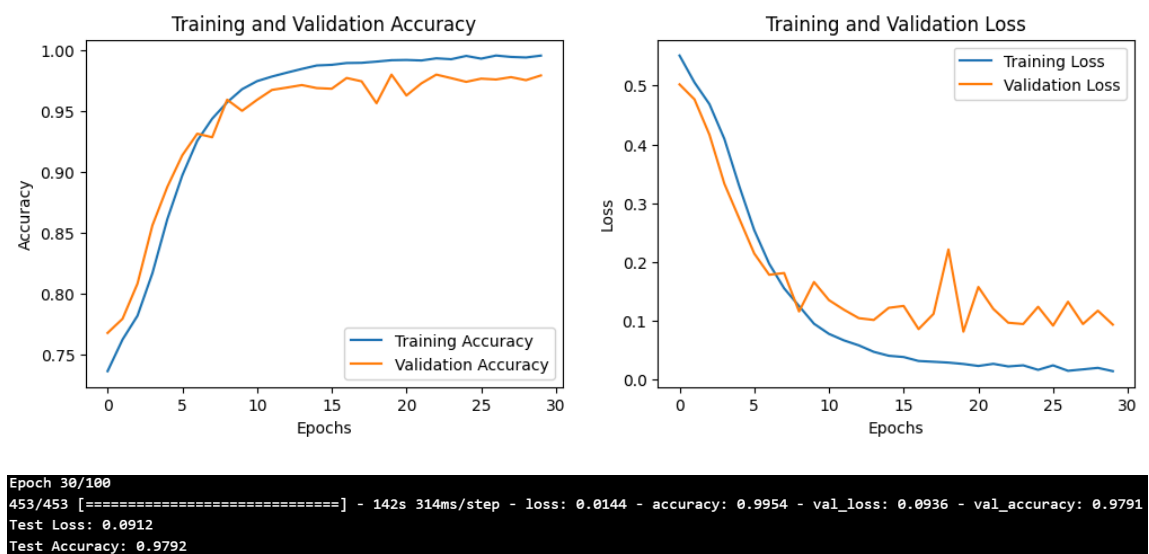
4.5.1 Learning Rate

This experiment focuses on experimenting learning rate with RNN GRU architecture, with learning rate values of 0.01, 0.001 and 0.0001 are tested on a 80:20 split.

Summary of learning rate Experiment

	EX1	EX2	EX3
Split	80/20	80/20	80/20
Dropout Layer (GRU)	0.3	0.3	0.5
Activation function (GRU)	Tanh	Tanh	Tanh
Dense Layer	128	128	128
Activation Function (Dense Layer)	ReLu	ReLu	ReLu
Dropout (Dense layer)	0.3	0.3	0.3
Activation Function (Output Layer)	sigmoid	sigmoid	sigmoid
Learning Rate	0.0001	0.01	0.001
Optimizer	Adam	Adam	Adam
Patience	10	10	10
Batch Size	64	64	64
Epoch	100	12	30
Test Accuracy	0.9394	0.8121	0.9792
Loss	0.1751	0.4124	0.0912
Precision	0.97	0.86	0.98
Recall	0.91	0.75	0.98
F1 Score	0.94	0.80	0.98

From the table it shows that a learning rate of 0.001 has achieved the highest accuracy, while a learning rate of 0.01 appears to be the lowest out of the three in terms of overall performance. The best model can be seen in figure



For the accuracy this model has achieved a very high accuracy of 99.54%, 97.91% and 97.92% and losses of 0.0144, 0.0935 and 0.0912 for training, validation and testing respectively. The model lasts for a total of 30 epochs, which shows the fast convergence of the model. In the graph however it can be seen that both training and validation follows closely with each other, which is validation accuracy and loss slightly diverges with its counterpart after the 9th epoch.

Confusion Matrix:

[[4498	26]
[162	4361]]

	precision	recall	f1-score
0	0.97	0.99	0.98
1	0.99	0.96	0.98

4.5.2 Dropout Layer

For the dropout layer experiment, both dropout rate from the GRU layers and dense layer are changed with the same value. The values being 0.3, 0.5 and 0.7 were tested on an 80:20 split.

Summary of dropout rate Experiment

	EX1	EX2	EX3
Split	80/20	80/20	80/20
Dropout Layer (GRU)	0.3	0.5	0.7
Activation function (GRU)	Tanh	Tanh	Tanh
Dense Layer	128	128	128
Activation Function (Dense Layer)	ReLu	ReLu	ReLu
Dropout (Dense layer)	0.3	0.5	0.7
Activation Function (Output Layer)	sigmoid	sigmoid	sigmoid
Learning Rate	0.001	0.001	0.001
Optimizer	Adam	Adam	Adam
Patience	10	10	10
Batch Size	64	64	64
Epoch	30	44	36
Test Accuracy	0.9792	0.9805	0.9691
Loss	0.0912	0.1053	0.1185
Precision	0.98	0.99	0.97
Recall	0.98	0.97	0.97
F1 Score	0.98	0.98	0.97

Table shows that each model performs relatively close to each other accuracy wise, with the dropout of 0.5 slightly triumphing over the other two. Dropout of 0.7 on the other hand only slightly lost to the other two models by the tiniest of margins. The figure below showcases the dropout rate of 0.5



```
Epoch 44/100
453/453 [=====] - 45s 100ms/step - loss: 0.0131 - accuracy: 0.9969 - val_loss: 0.1245 - val_accuracy: 0.9685
Test Loss: 0.1053
Test Accuracy: 0.9805
```

Figure shows that similar to other models during these experiments, it has exhibited a very high training accuracy of 99.69% and a training loss of 0.0131. Its validation accuracy and also showcases impressive numbers of 96.85% and 0.1245. The test accuracy on the other hand with the accuracy and loss of 98.05% and 0.1053 does not fall far off the other two. The model lasted for 44 epochs before getting stopped by early stopping where it exhibits consistent growth, except for a short spike lasting for an epoch or two around the 26th epoch.

Confusion Matrix:
[[4482 42]
[134 4389]]

precision	recall	f1-score
0.97	0.99	0.98
0.99	0.97	0.98

4.5.3 Optimizer

Rmsprop is also used to be experimented on as the optimizer rather than Adam for GRU, where the three different learning rates are changed to 0.01, 0.001 and 0.0001 to see how they perform on a 80:20 split.

Summary of optimizer Experiment

	EX1	EX2	EX3
Split	80/20	80/20	80/20
Dropout Layer (GRU)	0.3	0.3	0.5
Activation function (GRU)	Tanh	Tanh	Tanh
Dense Layer	128	128	128
Activation Function (Dense Layer)	ReLu	ReLu	ReLu
Dropout (Dense layer)	0.3	0.3	0.3
Activation Function (Output Layer)	sigmoid	sigmoid	sigmoid
Learning Rate	0.0001	0.01	0.001
Optimizer	Rmsprop	Rmsprop	Rmsprop
Patience	10	10	10
Batch Size	64	64	64
Epoch	100	18	52
Test Accuracy	0.9318	0.8340	0.9804
Loss	0.1807	0.3810	0.1101
Precision	0.94	0.80	0.99
Recall	0.93	0.89	0.97
F1 Score	0.93	0.84	0.98

Based on the experiments done by tweaking the learning rate of Rmsprop it has been found that the learning rate that achieved the highest accuracy is 0.001 on a 80:20 split. A learning rate of 0.01 however achieved the lowest accuracy compared to the others. Figure shows the results of the best performing model during the optimizer experiments.



```
Epoch 44/100
453/453 [=====] - 45s 100ms/step - loss: 0.0131 - accuracy: 0.9969 - val_loss: 0.1245 - val_accuracy: 0.9685
Test Loss: 0.1053
Test Accuracy: 0.9805
```

From the figure above it shows that this model which lasts for a total of 44 epochs has achieved a training accuracy of 99.69%, which is relatively close to the validation accuracy of 98.85%, with the validation loss for training and validation both being 0.0131 and 0.1245 respectively. The test accuracy has also achieved an accuracy of 98.05% and a loss of 0.1053. Furthermore for the training and validation loss, it can be seen that they follow the same trajectory, however validation loss later steers away from training loss around around the 10th epoch and then later on spikes way up at the end.

```
Confusion Matrix:
[[4482  42]
 [ 134 4389]]
```

precision	recall	f1-score
0.97	0.99	0.98
0.99	0.97	0.98

4.6 CNN versus RNN models

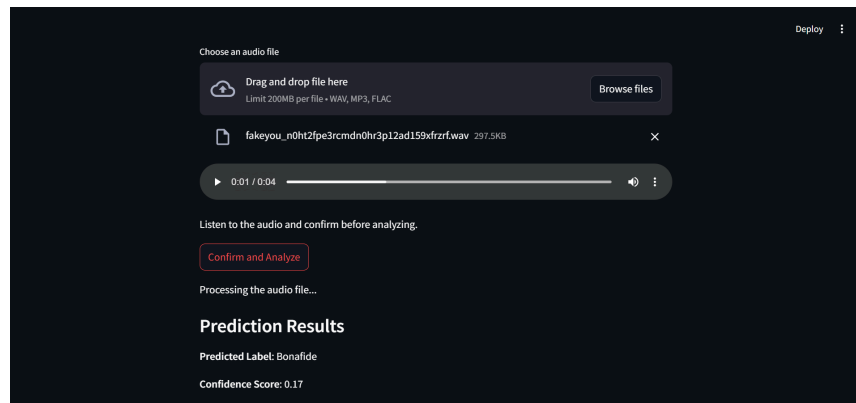
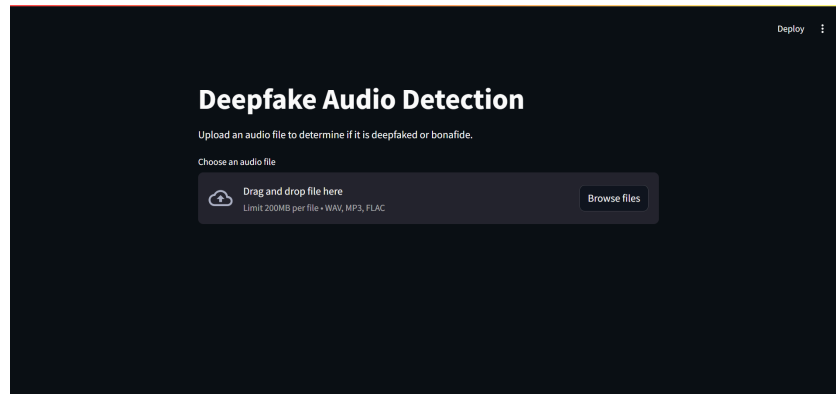
This section will compare the best model for both CNN and RNN model.

	CNN	LSTM	GRU
Data Split	80/20	80/20	80/20
Optimizer	Adam	RMSprop	Adam
Learning Rate	0.0001	0.001	0.001
Dropout Rate	0.4	0.3	0.3
Epoch	73	66	30
Activation function	Relu	tanh	tanh
Accuracy (Training, Validation, Testing)	99.47% 98.63% 98.13%	98.27% 94.32% 95.42%	99.54% 97.91% 97.92%
Loss (Training, Validation, Testing)	0.0138 0.0630 0.0454	0.0581 0.2111 0.1400	0.0144 0.0936 0.0912
Precision	0.99	0.96	0.98
Recall	0.98	0.95	0.98
F1 Score	0.99	0.95	0.98

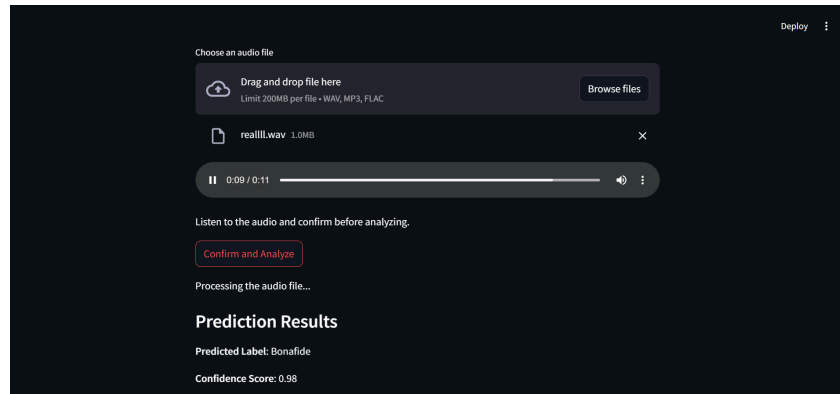
Based on the table above it can be seen that out of the three models CNN triumphs both LSTM and GRU by achieving the highest accuracy in validation and testing while also having the lowest loss across all phases . GRU however comes in at a close second where it barely outperforms CNN on training accuracy but falls behind in every other aspect. GRU however converges faster compared to the other two models as it only requires only 30 epochs compared to CNN's 73, LSTM however falls in behind in every aspect when compared with the other two models, despite still having a high testing accuracy of 95.42% it indicates that the other two models are more effective compared to LSTM. In conclusion, CNN appears to be the best performing model, while LSTM converges faster than both CNN and LSTM.

4.8 System Evaluation Results

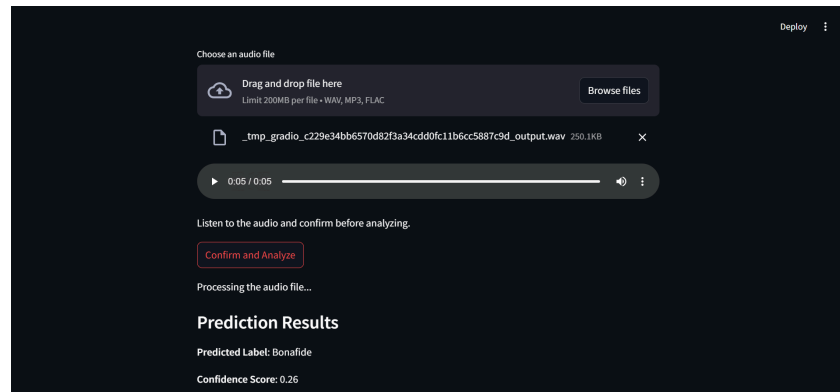
The prototype is developed using Streamlit where audio files can be uploaded into the prototype. This section will test the prototype with different ways audio can be deepfaked that were unseen.



The prototype is first tested on a deepfake audio that can be taken by simply googling for a deepfake voice generator from the fakeyou website where a tts can be generated on a celebrity voice, in this case a deepfaked voice of Donald Trump. The result shows that the model successfully predicted that the audio is indeed deepfake by having a low confidence score 0.17.



The second experiment consists of an audio file that contains the real voice of the researcher where the model has successfully predicted with 0.98 confidence score.



Lastly, the third experiment is conducted by utilising coqui which is an open source-platform that allows users to create custom TTS on new dataset using their pre-trained models. For this instance, a deepfaked audio of the researcher is used by utilising coqui where the model correctly predicts that the audio is deepfake with the confidence score of 0.26.

4.9 Summary

This chapter encapsulates how the experiments on both CNN and RNN were both conducted and how each model reacts towards the changes that have been made. The chosen CNN model were then tested with unseen data and with the prototype

CHAPTER 5

CONCLUSION AND RECOMMENDATION

This chapter summarizes the study's main conclusions and contributions, explains how each goal was accomplished, the advantages that were realized, the challenges encountered, and suggestions for future work to improve the system's efficacy. It also gives a summary of the project's accomplishments and looks at possible directions for future research and development.

5.1 Conclusion of the study

During this project it has successfully addressed and achieved the objectives intended. The first objective being to identify the key audio feature that effectively differentiate deepfaked audio recordings from genuine recordings which was achieved through the extensive literature review of research papers in chapter 2.

The second objective being to evaluate deep learning models to detect deepfaked audio was also fulfilled through extensive research in chapter 2 in selecting the appropriate techniques that is related to the domain, while also by doing an abundance amount of modelling and evaluation, as per chapter 3 and 4. The deep learning models that helped in achieving this objectives were CNN and RNN

Lastly, the last objective which is to develop a website based prototype for detecting audio was done by selecting the best model during the modelling phase and utilizing Streamlit to create the prototype. The said prototype is then able to reliably distinguish deepfaked audio from genuine ones.

5.3 Limitation of the study

The dataset that is being used for this study is from several years ago, 2021 to be precise and with the rapid advancements that are being made in audio deepfake and deepfake technology in general, the chosen dataset may not fully reflect or perform well on current techniques and characteristics of modern deepfakes. The dataset also does not take into account real-world audio such as background noises, bad recording devices and other various factors that can affect audio recordings, which limits the diversity of the dataset. Furthermore, the proposed system may not be able to cater for the needs of real-time or instant detection as a file must be first uploaded for the system to tell whether said audio file is spoofed or bonafide.

5.4 Recommendation and Future Work of the study

With the limitations that were identified, moving forward future work should focus on obtaining or updating datasets regularly to keep face with the advancements that are rapidly being made with audio deepfake technology. Furthermore, a system or application that could simultaneously detect the audio in real time would really benefit future work, as the need for real time detection of deepfake is crucial with the potential harm and misuse that comes with deepfake as a whole. For example an application that can detect whether an audio recording is spoofed or genuine by analyzing audio recordings using the device's microphone or audio from the system itself that can ensure real time detection.

5.5 Summary

In this chapter, the study discusses how it has achieved its primary objectives, the limitations and recommendations for future research. This study has successfully identified key audio features that effectively differentiate deepfaked audio recordings from genuine recordings, evaluate deep learning models to detect deepfaked audio and develop a website based prototype for detecting audio.

REFERENCES

- Abu Qais, Rastogi, A., Saxena, A., Rana, A., & Sinha, D. (2022). Deepfake Audio Detection with Neural Networks Using Audio Features. *2022 International Conference on Intelligent Controller and Computing for Smart Power (ICICCSPP)*. <https://doi.org/10.1109/icicccsp53532.2022.9862519>
- Almutairi, Z. M., & Hebah Elgibreen. (2023). Detecting Fake Audio of Arabic Speakers Using Self-Supervised Deep Learning. *IEEE Access*, 11, 72134–72147. <https://doi.org/10.1109/access.2023.3286864>
- Arora, D., Garg, M., & Gupta, M. (2020). Diving deep in Deep Convolutional Neural Network. *2020 2nd International Conference on Advances in Computing, Communication Control and Networking (ICACCCN)*. <https://doi.org/10.1109/icacccn51052.2020.9362907>
- Chaudhari, A., & Shedge, D. K. (2022). *Integration of CQCC and MFCC based Features for Replay Attack Detection*. <https://doi.org/10.1109/esci53509.2022.9758391>
- Dalibor Mitrović, Matthias Zeppelzauer, & Breiteneder, C. (2010). Features for Content-Based Audio Retrieval. *Advances in Computers*, 71–150. [https://doi.org/10.1016/s0065-2458\(10\)78003-7](https://doi.org/10.1016/s0065-2458(10)78003-7)
- Das, A., K.S Angel Viji, & Sebastian, L. (2022). *A Survey on Deepfake Video Detection Techniques Using Deep Learning*. <https://doi.org/10.1109/icngis54955.2022.10079802>
- Declaring the Value of Truth. (2019). *Journalism Practice*. <https://doi.org/10.1080/17512786.2018.1472526>

- Garg, D., & Gill, R. (2023). *Deepfake Generation and Detection - An Exploratory Study*. <https://doi.org/10.1109/upcon59197.2023.10434896>
- Guo, F., Lin, Z., Lyu, X., & Shi, Q. (2023). Does air pollution influence music sentiment? Measuring music sentiment by machine learning. *Journal of Asian Economics*, 87, 101617–101617. <https://doi.org/10.1016/j.asieco.2023.101617>
- H. Hakan Kilinc, & Faruk Kaledibi. (2023). *Audio Deepfake Detection by using Machine and Deep Learning*. <https://doi.org/10.1109/wincom59760.2023.10323004>
- Hamza, A., Abdul Rehman Javed, Iqbal, F., Kryvinska, N., Almadhor, A. S., Jalil, Z., & Rouba Borghol. (2022). Deepfake Audio Detection via MFCC Features Using Machine Learning. *IEEE Access*, 10, 134018–134028. <https://doi.org/10.1109/access.2022.3231480>
- Hartmann, K., & Giles, K. (2020). *The Next Generation of Cyber-Enabled Information Warfare*. <https://doi.org/10.23919/cycon49761.2020.9131716> <https://doi.org/10.13053/cys-25-2-3946>
- Islam Altalihin, Shadi AlZu'bi, Assal Alqudah, & Ala Mughaid. (2023). *Unmasking the Truth: A Deep Learning Approach to Detecting Deepfake Audio Through MFCC Features*. <https://doi.org/10.1109/icit58056.2023.10226172>
- Kaushal, A., Singh, S., Negi, S., & Shikhar Chhaukar. (2022). *A Comparative Study on Deepfake Detection Algorithms*. <https://doi.org/10.1109/icac3n56670.2022.10074593>

Khanjani, Z., Watson, G., & Janeja, Vandana P. (2021). *How Deep Are the Fakes? Focusing on Audio Deepfake: A Survey*. ArXiv.org.
<https://doi.org/10.48550/arXiv.2111.14203>

Khatri, N., Varun Borar, & Garg, R. (2023). *A Comparative Study: Deepfake Detection Using Deep-learning*.
<https://doi.org/10.1109/confluence56041.2023.10048888>

Kristiawan Nugroho, & Edy Winarno. (2022). *Spoofing Detection of Fake Speech Using Deep Neural Network Algorithm*.
<https://doi.org/10.1109/isemantic55962.2022.9920401>

Lalapura, V. S., Amudha, J., & Satheesh, H. S. (2022). Recurrent Neural Networks for Edge Intelligence. *ACM Computing Surveys*, 54(4), 1–38.
<https://doi.org/10.1145/3448974>

Malik, A., Minoru Kuribayashi, Abdullahi, S. M., & Ahmad Neyaz Khan. (2022). DeepFake Detection for Human Face Images and Videos: A Survey. *IEEE Access*, 10, 18757–18775.
<https://doi.org/10.1109/access.2022.3151186>

McAfee. (2023). *Beware the Artificial Impostor A McAfee Cybersecurity Artificial Intelligence Report*.
<https://www.mcafee.com/content/dam/consumer/en-us/resources/cybersecurity/artificial-intelligence/rp-beware-the-artificial-impostor-report.pdf>

Mekhail Mustak, Salminen, J., Matti Mäntymäki, Rahman, A., & Dwivedi, Y. K. (2023). Deepfakes: Deceptions, mitigations, and opportunities. *Journal of Business Research*, 154, 113368–113368.
<https://doi.org/10.1016/j.jbusres.2022.113368>

Mubarak, R., Tariq Alsboui, Alshaikh, O., Inuwa-Dute, I., Khan, S., & Parkinson, S. (2023). A Survey on the Detection and Impacts of Deepfakes in Visual, Audio, and Textual Formats. *IEEE Access*, 11, 144497–144529. <https://doi.org/10.1109/access.2023.3344653>

Neena Aloysius, & M. Geetha. (2017). *A review on deep convolutional neural networks*. <https://doi.org/10.1109/iccsp.2017.8286426>

Patel, Y., Sudeep Tanwar, Gupta, R., Bhattacharya, P., Innocent Ewean Davidson, Royi Nyameko, Srinivas Aluvala, & Vrince Vimal. (2023). Deepfake Generation and Detection: Case Study and Challenges. *IEEE Access*, 11, 143296–143323. <https://doi.org/10.1109/access.2023.3342107>

Pukale, D. D., S.G. Bhirud, & V.D. Katkar. (2017). *Content-based Image Retrieval using Deep Convolution Neural Network*. <https://doi.org/10.1109/iccube.2017.8463691>

R. Anagha, Arya, A., V. Hari Narayan, S. Abhishek, & T. Anjali. (2023). *Audio Deepfake Detection Using Deep Learning*. <https://doi.org/10.1109/smart59791.2023.10428163>

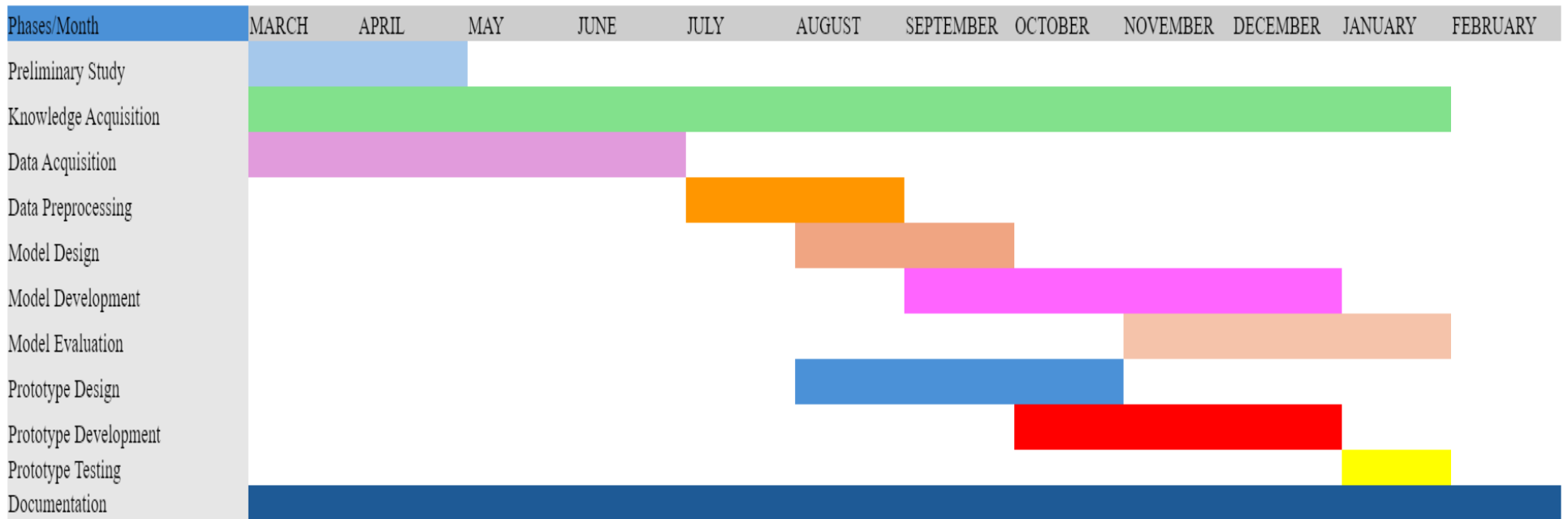
R. L. M. A. P. C. Wijethunga, D. M. K. Matheesha, A. A. Noman, K. H. V. T. A. De Silva, M. Tissera and L. Rupasinghe, "Deepfake Audio Detection: A Deep Learning Based Solution for Group Conversations," 2020 2nd International Conference on Advancements in Computing (ICAC), Malabe, Sri Lanka, 2020, pp. 192-197, doi: 10.1109/ICAC51239.2020.9357161.

Saranga Kingkor Mahanta, Abdullah, & Partha Pakray. (2021). Deep Neural Network for Musical Instrument Recognition Using MFCCs. *Computación Y Sistemas*, 25(2).

- Shaaban, O. A., Yildirim, R., & Abubaker Alguttar. (2023). Audio Deepfake Approaches. *IEEE Access*, *11*, 132652–132682. <https://doi.org/10.1109/access.2023.3333866>
- Shi, L., Li, C., & Tian, L. (2019). *Music Genre Classification Based on Chroma Features and Deep Learning*. <https://doi.org/10.1109/icicip47338.2019.9012215>
- Shoaib, M. R., Wang, Z., Milad Taleby Ahvanooey, & Zhao, J. (2023). *Deepfakes, Misinformation, and Disinformation in the Era of Frontier AI, Generative AI, and Large AI Models*. <https://doi.org/10.1109/icca59364.2023.10401723>
- Stamatis Karnouskos. (2020). Artificial Intelligence in Digital Media: The Era of Deepfakes. *IEEE Transactions on Technology and Society*, *1*(3), 138–147. <https://doi.org/10.1109/tts.2020.3001312>
- Uddin, B., Sharmin, A., & Feni, B. (2020). *D Deep Insights of Deepfake Technology : A Review*. <https://doi.org/10.48550/arXiv.2105.00192>
- Wahl-Jorgensen, Karin & Carlson, Matt. (2021). Conjecturing Fearful Futures: Journalistic Discourses on Deepfakes. *Journalism Practice*. *15*. 1-18. <https://doi.org/10.1080/17512786.2021.1908838>
- Zeinali, H., Stafylakis, T., Athanasopoulou, G., Rohdin, J., Gkinis, I., Burget, L., & Černocký, J. H. (2019). *Detecting Spoofing Attacks Using VGG and SincNet: BUT-Omilia Submission to ASVspoof 2019 Challenge*. ArXiv.org. <https://doi.org/10.48550/arXiv.1907.12908>

- Zhou, X., & Zafarani, R. (2020). A Survey of Fake News: Fundamental Theories, Detection Methods, and Opportunities. *ACM Computing Surveys*, 53(5). <https://doi.org/10.1145/3395046>
- Narwane, S. V., & Sawarkar, S. D. (2022). Is handling unbalanced datasets for machine learning uplifts system performance?: A case of diabetic prediction. *Diabetes & Metabolic Syndrome Clinical Research & Reviews*, 16(9), 102609–102609. <https://doi.org/10.1016/j.dsx.2022.102609>
- Daneshjou, R., He, B., Ouyang, D., & Zou, J. Y. (2021). How to evaluate deep learning for cancer diagnostics – factors and recommendations. *Biochimica et Biophysica Acta (BBA) - Reviews on Cancer*, 1875(2), 188515–188515. <https://doi.org/10.1016/j.bbcan.2021.188515>
- Jeyabharathy Sadaiyandi, Arumugam, P., Arun Kumar Sangaiah, & Zhang, C. (2023). Stratified Sampling-Based Deep Learning Approach to Increase Prediction Accuracy of Unbalanced Dataset. *Electronics*, 12(21), 4423–4423. <https://doi.org/10.3390/electronics12214423>
- Liu, X., Wang, X., Md Sahidullah, Patino, J., Delgado, H., Kinnunen, T., Todisco, M., Yamagishi, J., Evans, N., Nautsch, A., & Lee, K. A. (2023). ASVspoof 2021: Towards Spoofed and Deepfake Speech Detection in the Wild. *IEEE/ACM Transactions on Audio Speech and Language Processing*, 31, 2507–2522. <https://doi.org/10.1109/taslp.2023.3285283>

APPENDICES



MODEL	ACCURACY			RECALL	PRECISION	F1-SCORE
	TRAINING	VALIDATION	TESTING			
CNN	99.47%	98.63%	98.13%	0.99	0.98	0.99
LSTM	98.27%	94.32%	97.91%	0.96	0.95	0.95
GRU	99.54%	97.91%	97.92%	0.98	0.98	0.98